In-Home Assistive Robot

User Stories and Use Cases

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI assisted with LaTeX formatting.

Project: In Home Assistive Robot / remote-care-bot

Our robot is a compact two-wheel robot with a front-facing screen. It lets a family caregiver have a video conversation with an older or disabled adult at home, and it can follow the resident from room to room so the conversation doesn't have to end when they get up. The resident decides when calls start, when the robot follows, and when it stops.

1 Stakeholder map

Category	Specific roles	Needs
Primary	Older or disabled adult who lives alone or spends much of the day alone	Keep talking with family while moving around the home; decide who can see them and when. US-01, US-02.
Secondary	Remote family caregiver (for example, an adult child in another city)	Understand what actually happened when a call doesn't connect, without guessing about the resident's health. US-03.
Hidden	Other people in the home, such as a spouse, roommate or visitor	Not be followed, filmed or bumped into by a robot that isn't there for them. US-04.
Hidden	Residents who use a wheelchair or have limited reach or dexterity	Use the robot's controls and screen from a seated position. US-05.

2 User stories

US-01 - Keep a call going while moving to another room

Category: Primary - older adult who lives alone. **Story:** As an older adult on a video call with my daughter, I want the call to come with me when I walk to another room, so that I don't have to end the conversation or shout across the house.

INVEST	Self-check
Independent	Builds on an accepted call but delivers its own value: the conversation survives a room change.
Negotiable	How the resident asks the robot to follow is left open.
Valuable	The conversation continues instead of being cut short.
Estimable	One short route on one floor at the robot's tested speed of 0.2 m/s.
Small	One resident-requested room change; stairs and remote driving are excluded.
Testable	UC-01 and AC-01.1 to AC-01.3.

US-02 - Decide before the camera turns on

Category: Primary - older adult who lives alone. **Story:** As an older adult who lives alone, I want to accept or decline each caregiver call before my camera and microphone turn on, so that no one can see or hear into my home without my say-so.

INVEST	Self-check
Independent	Works with the robot stationary and without following.
Negotiable	How the call is announced and answered is left open.
Valuable	The resident stays in control of their own privacy.
Estimable	One incoming call from one named caregiver.
Small	Accept, decline or end a single call.
Testable	AC-01.1 and AC-02.2.

US-03 - Understand why a call didn't connect

Category: Secondary - remote family caregiver. **Story:** As a family caregiver who lives in another city, I want to know whether my parent declined my call, didn't answer, or the robot lost its connection, so that I don't mistake a dropped call for my parent being fine or being in trouble.

INVEST	Self-check
Independent	Can be tested with simulated call outcomes, without the robot moving.
Negotiable	Wording and layout of each outcome are left open, as long as each one looks different.
Valuable	The caregiver can decide whether to try again, call another way, or wait.
Estimable	Four outcomes: connected, declined, no answer, connection lost.
Small	One call attempt; emergency alerts and health monitoring are excluded.
Testable	UC-02 and AC-02.1 to AC-02.3.

US-04 - Don't get followed by mistake

Category: Hidden - spouse, roommate or visitor in the home. **Story:** As a spouse who shares the home, I want the robot to stop rather than start following me when I walk between it and my partner, so that it doesn't trail me around the house or run into me.

INVEST	Self-check
Independent	Can be tested with recorded camera footage before any motors are involved.
Negotiable	How the robot tells people apart is left open.
Valuable	Other people in the home aren't followed or bumped.
Estimable	One other person crossing the robot's view.
Small	One crossing event during one follow.
Testable	AC-01.3.

US-05 - Reach the controls from a chair

Category: Hidden - resident who uses a wheelchair. **Story:** As a resident who uses a wheelchair, I want to reach the robot's stop and privacy controls from my seat, so that I can end a call or stop the robot without standing up or asking for help.

INVEST	Self-check
Independent	Can be tested on a bench mockup with no call or movement.
Negotiable	Where the controls go is left open; the screen sits only about 24 cm above the floor today, so they may need to be separate from it.
Valuable	The resident keeps control without depending on someone else.
Estimable	Two controls tested from one seated position.
Small	Reach and press only; screen readability is a separate story.
Testable	A seated tester presses stop and privacy without leaning out of the chair, in 3 of 3 tries.

3 Use cases

UC-01 - Continue a caregiver call into another room

Linked story: US-01 (also covers US-02 and US-04). **Primary actor:** Resident. **Secondary actors:** Remote caregiver; other people in the home.

Preconditions:

1. Caregiver C has the resident's permission to call, and FOLLOW is turned off.
2. The robot is parked next to the resident on a cleared single-floor route.
3. Both devices are online and send a heartbeat once per second.

Main success flow:

1. **Caregiver:** Calls the resident.
2. **System:** Announces the call with C's name. The camera and microphone stay off.
3. **Resident:** Accepts the call.
4. **System:** Starts the call and shows that the camera and microphone are on.
5. **Resident:** Asks the robot to follow and walks toward another room.
6. **System:** Follows the resident at no more than 0.2 m/s and keeps the call going.
7. **Resident:** Stops in the new room.
8. **System:** Stops within 1 m of the resident and keeps the call going.
9. **Resident:** Ends the call.
10. **System:** Turns off the camera and microphone, turns FOLLOW off, and shows Ended.

Alternate flow A1 - Resident declines: At step 3, the resident declines. The system tells C the call was declined, and the camera and microphone never turn on.

Exception flow X1 - The robot loses the resident: During step 6, the resident leaves the camera's view or another person walks between them. The robot stops and stays stopped, the call continues from where it stopped, and the screen asks the resident to restart following. It never starts following someone else on its own.

Postcondition: The call has ended, the robot is stopped and FOLLOW is off. No audio or video was recorded.

UC-01 acceptance criteria

1. **AC-01.1 - Consent before camera (main flow). Given** the UC-01 preconditions, **When** C calls, **Then** the resident sees C's name within 2 seconds, and the camera and microphone stay off until the resident accepts.
2. **AC-01.2 - Following during a call (main flow). Given** an active call with FOLLOW on, **When** the resident walks 5 m to another room and stops, **Then** the robot never exceeds 0.2 m/s, stops within 1 m of the resident, and the call stays connected the whole time.
3. **AC-01.3 - Target lost (X1). Given** the robot is following during a call, **When** the resident leaves the camera's view or another person crosses between them, **Then** the robot's travel request drops to zero within 1 second, it settles within 25 mm of where it lost the resident, its speed stays under 5 mm/s for a full second within 10 seconds of the loss, the call stays connected, and the robot doesn't move again until the resident restarts following.

UC-02 - Find out why a call didn’t connect

Linked story: US-03. **Primary actor:** Remote caregiver. **Secondary actors:** Resident; the robot.

Preconditions:

1. Caregiver C has the resident’s permission to call.
2. The robot is online and no call is active.

Main success flow:

1. **Caregiver:** Calls the resident.
2. **System:** Shows C that the robot is ringing.
3. **Resident:** Accepts the call.
4. **System:** Shows C that the call is connected.

Alternate flow A1 - No answer: At step 3, the resident doesn’t respond for 60 seconds. The system stops ringing and shows C ”No answer”. It doesn’t suggest anything about the resident’s health.

Exception flow X1 - Connection lost: At any step, no heartbeat arrives from the robot for 5 seconds. The system ends the attempt and shows C ”Robot offline” with the time it was last online.

Postcondition: C sees exactly one outcome: Connected, Declined, No answer or Robot offline.

UC-02 acceptance criteria

1. **AC-02.1 - Connected (main flow). Given** the UC-02 preconditions, **When** the resident accepts, **Then** C sees Connected within 2 seconds.
2. **AC-02.2 - Declined (US-02). Given** the robot is ringing, **When** the resident declines, **Then** C sees Declined within 2 seconds, and the resident’s camera and microphone never turned on.
3. **AC-02.3 - Robot offline (X1). Given** the robot is ringing or connected, **When** 5 seconds pass without a heartbeat, **Then** C sees Robot offline and the time it was last online, and never sees Connected or No answer for that attempt.

In-Home Assistive Robot

High-Level System Design

Design D0

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

October 2, 2026

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI was used to format this document in LaTeX.

1 Title, goal statement, and conventions

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

The basic input to the system is a caregiver's call request, the resident's choices, and readings from the robot's sensors. The basic output is a two-way video call that the resident has accepted, a clear display of the call and privacy status, and robot movement that either follows the resident or stops.

We are designing and building the robot's physical platform ourselves (the chassis, mounts, enclosure, and mechanisms) and fitting it with purchased motors, sensors, electronics, and a screen. For this first design we are focusing on calls and on following the resident when they ask, in a controlled, single-floor indoor space. We have also talked about item delivery, reminders, an adjustable screen, and help detection, but have not decided on those yet.

In each diagram, blue rounded boxes are parts we build and gray dashed boxes are outside services we rely on. Stick figures are people, and dashed blue outlines show where the software runs. Arrows point in the direction information or control moves, and the I1-I16 labels match the rows of the interface table. The red line is the planned direct wire for the Stop and Privacy buttons, and the small open-ended boxes are places where information is stored.

Every part traces back to at least one of our five user stories. Keeping a call going while the resident moves (US-01) uses the whole system. The resident controls and the call manager make sure nothing starts without the resident's consent (US-02). The call server and caregiver app tell the caregiver what happened to a call (US-03). Person tracking, motion control, and the platform keep the robot from following the wrong person (US-04). The resident controls, and where we place them, make the robot usable from a seat (US-05).

2 Block diagram (the D0 diagram)

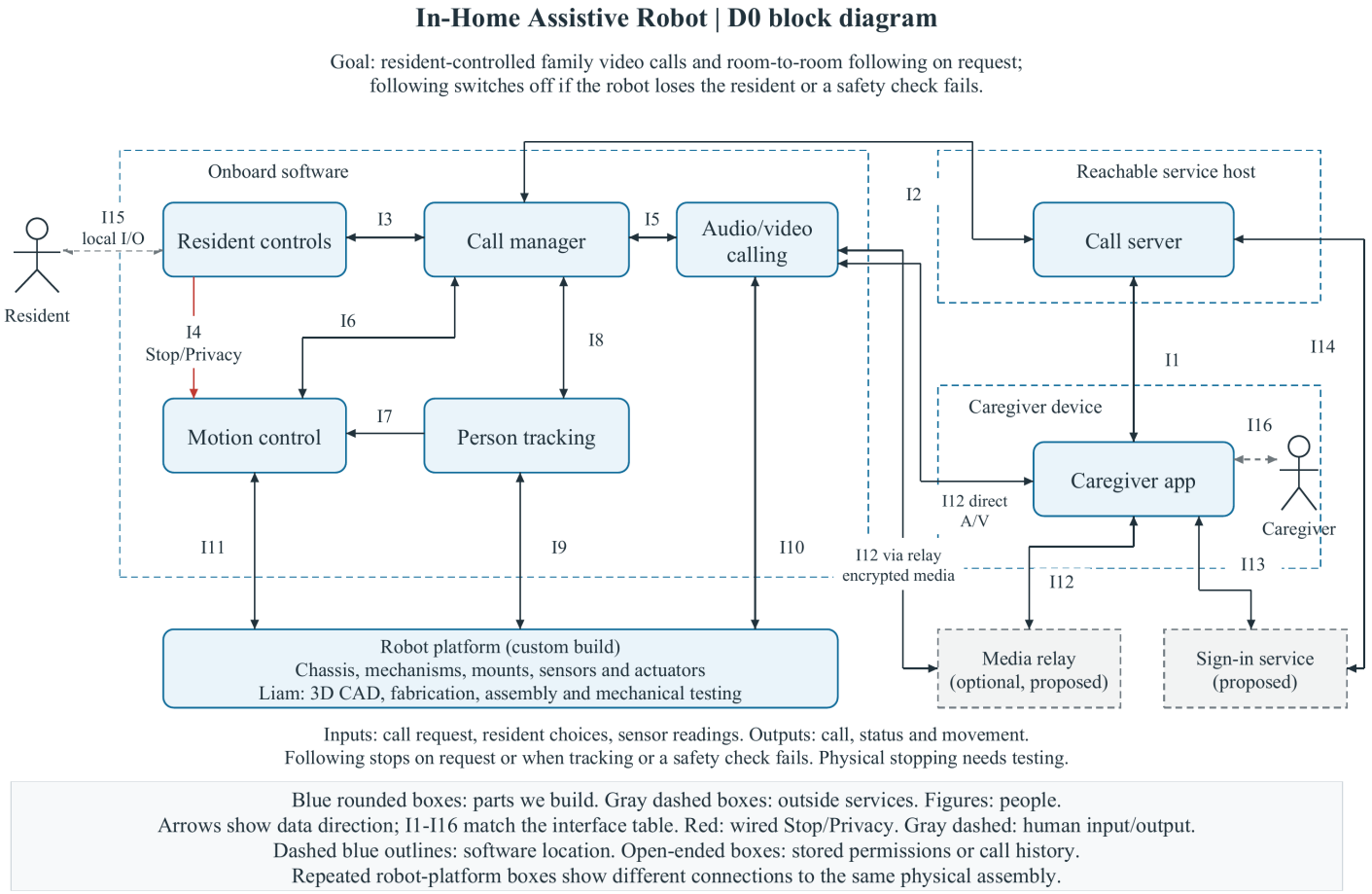


Figure 1. System block diagram

The robot's software runs on the platform we are building, while the call server and caregiver app run separately. We plan to use an outside sign-in service and, when a direct connection fails, a media relay, but we have not picked providers for either yet.

3 Component responsibility table

Liam is in charge of the mechanical build: 3D modeling, chassis and mechanism design, fabrication, assembly, and mechanical testing. He will also plan test fixtures and check fit, strength, clearances, and low-speed driving. Gavin is in charge of the robot's form, where the screen and controls go, and the experience for both the resident and the caregiver. Austin is in charge of the software, communication, sensing, and control code, and works with Liam where the mechanical and electrical parts meet.

Table 1. Responsibilities for the eight parts of the system

Component and responsibility	Interfaces	Owner and stories
Resident controls: Gives the resident the screen and buttons for calls, privacy, and following.	In: I3, I15. Out: I3, I4, I15	Gavin Lamphier. US-01, US-02, US-04, US-05
Call manager: Decides what the call and the robot are allowed to do, based on the resident's choices.	In: I2, I3, I5, I6, I8. Out: I2, I3, I5, I6, I8	Austin Heiss. US-01, US-02, US-03, US-04, US-05
Audio/video calling: Sends and receives live audio and video once the resident accepts a call.	In: I5, I10, I12. Out: I5, I10, I12	Austin Heiss. US-01, US-02, US-03, US-05
Person tracking: Reports where the selected resident is, or that it can't find them.	In: I8, I9. Out: I7, I8, I9	Austin Heiss. US-01, US-02, US-04, US-05
Motion control: Turns the resident's position into movement commands, requesting a stop whenever it should.	In: I4, I6, I7, I11. Out: I6, I11	Liam Patel. US-01, US-04, US-05
Robot platform: Is the physical robot that carries the sensors, screen, speaker, and motors.	In: I9, I10, I11. Out: I9, I10, I11	Liam Patel. US-01, US-04, US-05
Call server: Keeps track of each call attempt so the caregiver and the robot agree on its status.	In: I1, I2, I14. Out: I1, I2, I14	Austin Heiss. US-01, US-02, US-03
Caregiver app: Lets the caregiver start or end a call and see what happened to it.	In: I1, I12, I13, I16. Out: I1, I12, I13, I16	Gavin Lamphier. US-01, US-03

4 Interface specification table

Each row lists the main fields and their types, the data format, how the data is sent, and which part handles a failure. We plan for the parts on the robot to talk through function calls inside one program; we still need to choose the physical connections and the outside service providers. A call's status is one of ringing, connected, declined, no answer, offline, ended, or failed.

Table 2. Interface specifications

Connection	Specification
I1 caregiver app <-> call server	**Inputs and outputs.** Request: robot_id and access_token (text), action (start or end). Response: call_id (text), status (call status), last_seen (timestamp or none). **Data format.** JSON **Protocol.** HTTPS requests; secure WebSocket status updates **Error behavior and handler.** The call server rejects bad requests, and the app tells the caregiver when it can't connect.

Table 2. Interface specifications

Connection	Specification
I2 call server <-> call manager	**Inputs and outputs.** Invitation: call_id and caller_name (text). Response: call_id (text), status (call status); robot_online (boolean). **Data format.** JSON **Protocol.** Secure WebSocket; robot sends status each second **Error behavior and handler.** The server reports the robot offline after 5 seconds without an update; if the connection drops during a call, the call manager ends the call.
I3 resident controls <-> call manager	**Inputs and outputs.** Input: action (accept, decline, end, follow, stop, or privacy), target_id (text or none). Output: caller_name (text), call_status and follow_status (status). **Data format.** Typed in-memory records **Protocol.** Proposed in-process request and status calls **Error behavior and handler.** The call manager ignores actions that don't make sense at that moment, and the controls show an error.
I4 resident controls -> motion control	**Inputs and outputs.** Input: stop and privacy (on/off). Status returns on I6 and I3. **Data format.** Digital signal **Protocol.** Proposed direct wired contacts **Error behavior and handler.** Motion control requests a stop when Stop or Privacy is pressed, or if the wire is broken.
I5 call manager <-> audio/video calling	**Inputs and outputs.** Input: capture_allowed (boolean). Output: media_status (connecting, connected, or failed). **Data format.** Typed in-memory records **Protocol.** Proposed in-process permission and status calls **Error behavior and handler.** Audio/video calling stops capture if permission is lost; the call manager reports failure.
I6 call manager <-> motion control	**Inputs and outputs.** Input: follow_requested (boolean). Output: motion_status (following, stopping, stopped, or fault). **Data format.** Typed in-memory records **Protocol.** Proposed in-process request and status calls **Error behavior and handler.** Motion control requests a stop if following is no longer allowed or requested, and the call manager shows the new status.
I7 person tracking -> motion control	**Inputs and outputs.** Output: target_id (text), range_m and bearing_deg (numbers or none), tracking_status (valid or unavailable). **Data format.** Typed in-memory record **Protocol.** Proposed in-process position update **Error behavior and handler.** Motion control requests a stop if tracking becomes unavailable, then waits until the resident asks it to follow again.

Table 2. Interface specifications

Connection	Specification
I8 call manager <-> person tracking	**Inputs and outputs.** Input: tracking_allowed (boolean), target_id (text). Output: tracking_status (valid or unavailable). **Data format.** Typed in-memory records **Protocol.** Proposed in-process permission and status calls **Error behavior and handler.** If tracking is no longer allowed, person tracking stops and reports itself as unavailable.
I9 robot platform <-> person tracking	**Inputs and outputs.** Input: capture_requested (boolean). Output: image (image frame), distance_m (numbers), sensor_status (available or unavailable). **Data format.** Image and numeric sensor data **Protocol.** Local sensor-driver calls; physical connections to be selected **Error behavior and handler.** If the sensors aren't working, person tracking reports them as unavailable and doesn't give a position.
I10 robot platform <-> audio/video calling	**Inputs and outputs.** Input: camera_frame (image), mic_audio (audio). Output: display_frame (image), speaker_audio (audio), capture_requested (boolean). **Data format.** Image and audio buffers; typed control record **Protocol.** Local camera, audio, and display driver calls; physical connections to be selected **Error behavior and handler.** If the camera, microphone, speaker, or display fails, audio/video calling reports it and ends the call.
I11 motion control <-> robot platform	**Inputs and outputs.** Output: speed_m_s and turn_rate_deg_s (numbers). Input: measured_speed_m_s (number), path_clear (boolean), platform_status (available or fault). **Data format.** Numeric control and sensor records **Protocol.** Local motor and sensor driver calls; physical connections to be selected **Error behavior and handler.** Motion control requests a stop if sensor feedback stops arriving. We still need to test how well the robot actually stops.
I12 audio/video calling <-> caregiver app; optional media relay	**Inputs and outputs.** Both directions: video (video stream), audio (audio stream). **Data format.** Encoded audio/video packets **Protocol.** Proposed encrypted WebRTC, directly or through a relay **Error behavior and handler.** Audio/video calling reports when the connection is lost, and the caregiver app shows that the call failed.
I13 caregiver app <-> sign-in service	**Inputs and outputs.** Input: sign_in (credential text). Output: authenticated (boolean), access_token (text). **Data format.** Sign-in form and JSON response **Protocol.** HTTPS **Error behavior and handler.** The caregiver app shows that sign-in failed and won't place calls until the caregiver signs in.

Table 2. Interface specifications

Connection	Specification
I14 call server <-> sign-in service	**Inputs and outputs.** Input: access_token (text). Output: valid (boolean), caregiver_id (text). **Data format.** JSON **Protocol.** HTTPS verification request **Error behavior and handler.** If the call server can't verify a request, it rejects it and reports a service error.
I15 resident <-> resident controls	**Inputs and outputs.** Input: action (touch/button choice). Output: caller_name (text), privacy, following, camera, and microphone (on, off, or unknown), alert (sound). **Data format.** Touch/button events; text, indicators, and sound **Protocol.** Local button and screen interaction **Error behavior and handler.** Resident controls show an error if an action cannot be completed.
I16 caregiver <-> caregiver app	**Inputs and outputs.** Input: action (call or end). Output: call_status (call status), video (video stream), audio (audio stream). **Data format.** Touch/click events; text and audio/video **Protocol.** Phone or browser interaction **Error behavior and handler.** The caregiver app displays an unavailable or failed status when a call cannot connect.

For example, a caregiver starting a call on I1 would send the robot's ID, their sign-in token, and the action:

```
{"robot_id": "home-robot-01", "access_token": "example", "action": "start"}
```

5 Data-flow diagram

In the call flow, the caregiver's request goes through the call server to the robot. The resident sees who is calling and decides whether to answer. Once they accept, live audio and video go between the robot and the caregiver, and the call status comes back through the server. In this first design, the camera and microphone are only used during a call the resident has accepted.

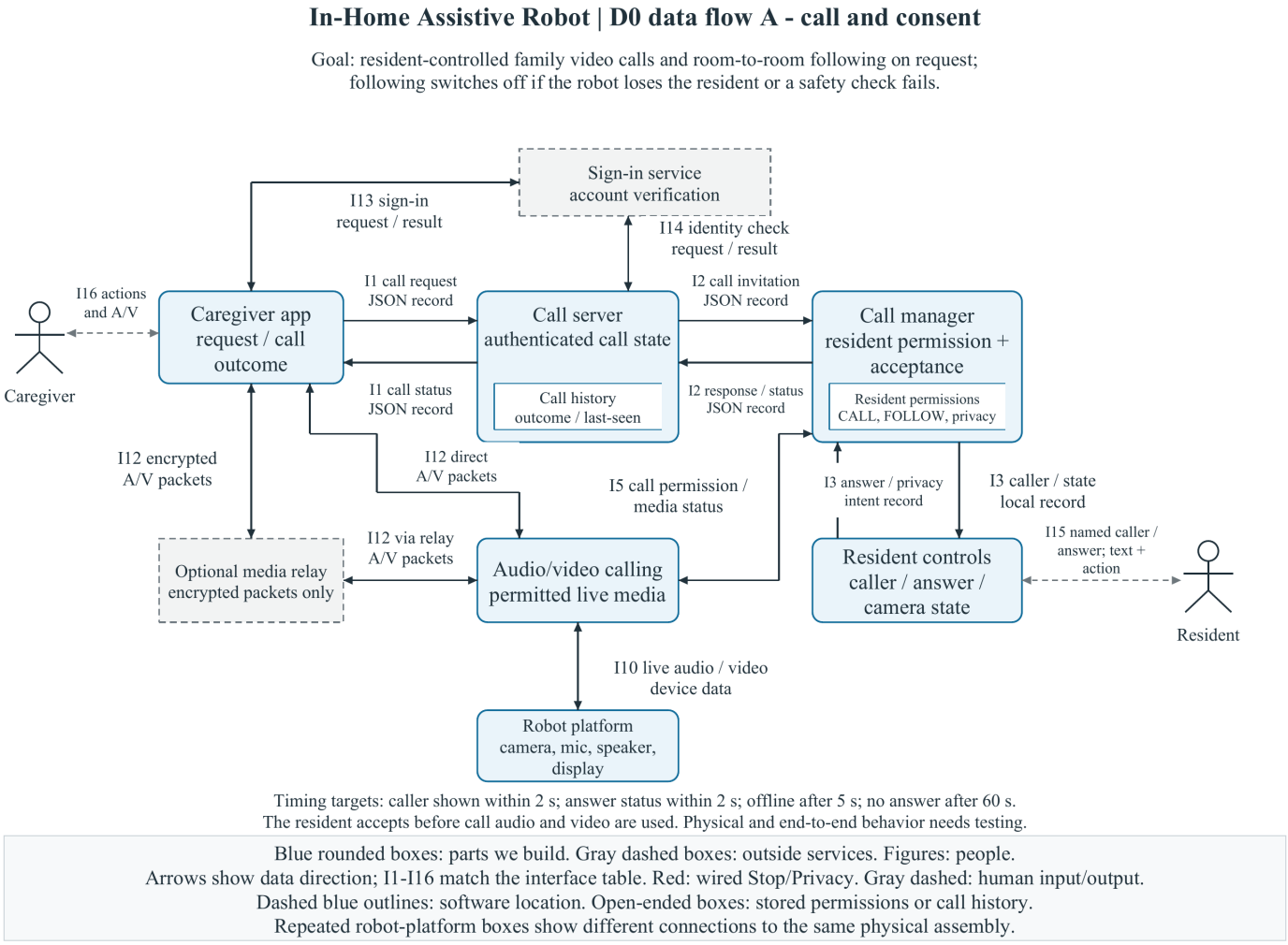


Figure 2. Call and consent data flow

In the following flow, the resident asks the robot to follow them. The robot's sensor readings are turned into the selected person's position, and motion control uses that position along with the robot's safety checks to decide how to move. If the robot loses the person, the resident turns following off, or a safety check fails, motion control requests a stop. After losing the person, the robot won't start following again until the resident asks, but the call can keep going while the robot is stopped. We still have to test how well the robot actually stops once it is built.

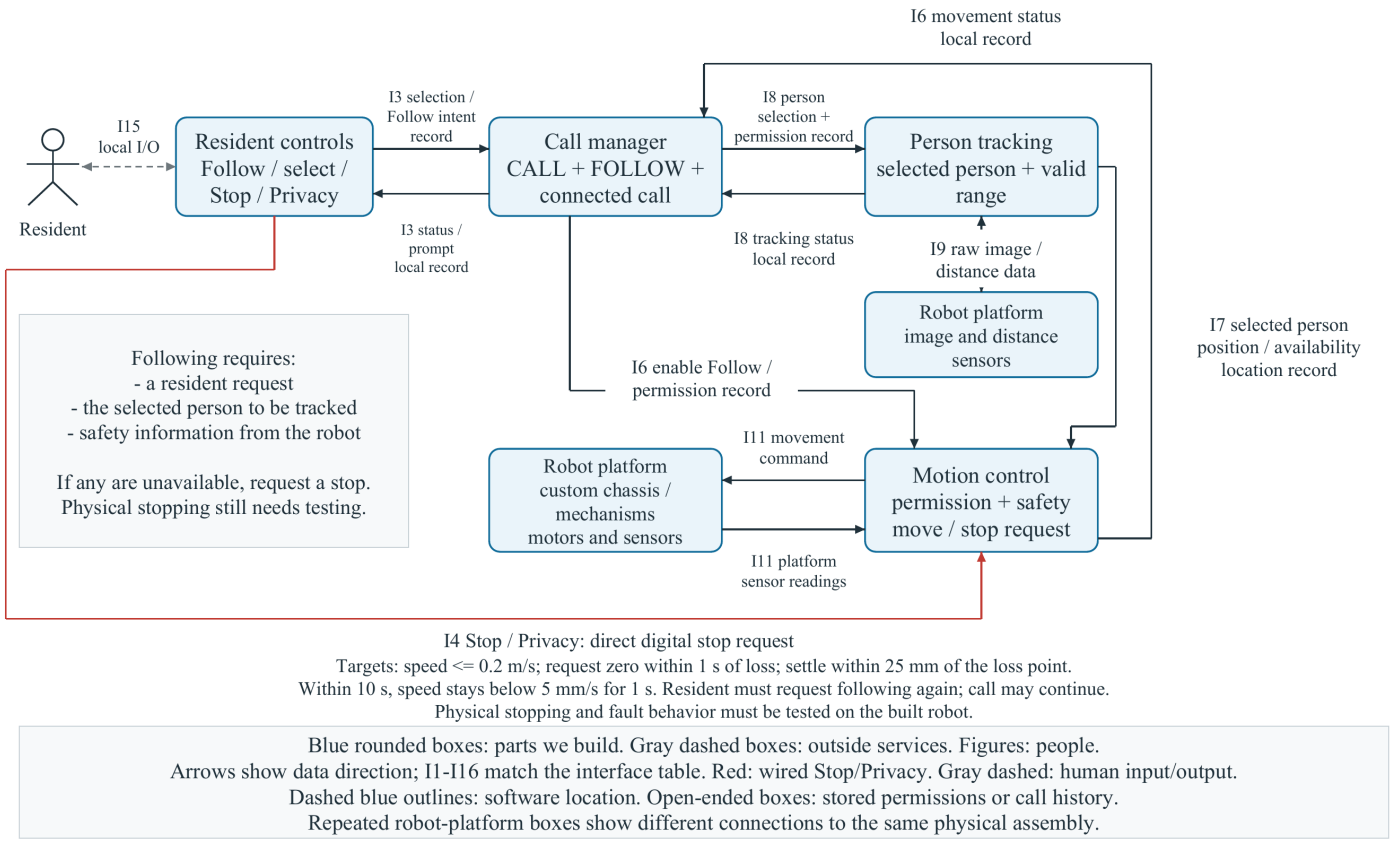


Figure 3. Following and stop data flow

Our user stories set a maximum following speed of 0.2 m/s and require the movement request to drop to zero within 1 second of losing the person. After that, the robot has to settle within 25 mm of where it lost the resident and stay below 5 mm/s for a full second, all within 10 seconds. These are targets for the design; we haven't shown them on the physical robot yet.

6 Architecture pattern and justification

We use a **client-server** pattern for communication: the caregiver app and the robot both connect to a call server that keeps track of each call's status. On the robot, a **layered** design separates the resident controls, the call manager, and the services underneath that handle communication and movement. Following works as a **pipeline** that goes from sensor readings to a position estimate to a movement command. Finally, the motion hardware follows an **embedded sensor-actuator** pattern so that movement control stays on the robot.

Fit to the problem. Our project combines remote communication with a physical robot, so we need a server for the communication side and local control for the robot side. These four patterns cover both.

Team skills. The split matches what each of us is good at: Gavin leads interaction design, Liam leads the mechanical build and testing, and Austin leads software and sensing.

Performance and timing. The robot has to react to a lost person or a Stop press even if the internet goes down, so movement and stopping are handled on the robot itself. We still need to measure the timing and stopping behavior on the built robot.

Scalability. One call server is enough for our first robot and caregiver, and it could support more users later if needed. We decided against **microservices** for the first build, because running several separate services would add a lot of setup and maintenance without a real benefit at our scale.

Hardware constraints. The robot's size, battery, computing power, and cost all limit our hardware choices, which are still open. Our build and hosting budgets aren't approved yet. Before choosing a provider, we will compare a parts and fabrication estimate (with some contingency) against our available funds, and look at ongoing hosting and support costs.

Privacy and accessibility also shaped the design. The resident decides when to accept a call, when the robot follows, and when privacy mode is on, and the controls have to be usable from a seated position. The first version doesn't record any audio or video. During design and physical testing, we will look at where the camera and screen go, how the privacy controls work, and whether the robot creates a tripping hazard. If we add help detection later, we still need to work out how it could respect the resident's privacy.

7 Decision log

1. **Build the platform ourselves.** We considered buying a complete commercial base, but that would leave out most of our mechanical design work. Building a custom chassis lets Liam model, fabricate, assemble, and test the robot around the components we pick.

2. **Start with calling and requested following.** We also considered item delivery, reminders, and help detection. Calling and following give us one clear scenario for building out the communication and mobility platform, and we can still add the other features later.

3. **Use a call server.** We considered connecting the app and robot directly (peer-to-peer only), but then nothing could tell the caregiver the robot is offline when it drops off the network. A small server gives the app and the robot one place to exchange call requests and status.

4. **Keep movement control on the robot.** Sending movement decisions through a remote service would make the robot depend on the network to move or stop. Keeping control local means it can react quickly when it loses the person or someone presses Stop.

5. **Require resident choice for calls and following.** We considered starting calls or following automatically, but that could surprise the resident and raise privacy concerns. Instead, the resident has to accept each call and separately ask the robot to follow.

6. **Use an initial fixed screen and sensor mounts.** An extending screen or a moving sensor might give a better view, but it adds mechanical and integration work. A fixed setup is easier to build and test first, and we can decide later whether an adjustable screen is worth it.

In-Home Assistive Robot

Detailed System Design

Design D1

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

October 8, 2026

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI assisted with drafting and LaTeX formatting.

1 Header, scope, and conventions

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

This document details the **call server** and **person tracking**; the resident controls, call manager, audio/video calling, motion control, robot platform, and caregiver app are deferred to D2. We picked these two because the call server holds the data model and person tracking is the computation that breaks following if it is slow or wrong. The APIs in Section 5 also cover where they meet their neighbors (I1, I2, I7, I8, and I14). The caregiver app appears here only as the client of I1 and as a technology choice in Section 6.

Figure conventions. Figure 1 is the only figure. Each blue box is an entity, with its name in bold and its key attributes below it. Each solid line is a relationship; lines have no arrows because they are not data flow. PK marks a primary key, FK a foreign key, and UQ a field covered by a unique index. Cardinality is written at both ends of every line: **1** means exactly one and **0..*** means zero or more. The number at a line end tells how many of the entity next to it belong to one record at the other end; for example, the **0..*** next to Pairing means one robot can have zero or more pairings. The legend repeats this on the figure.

This is a proposed build plan for one resident, one caregiver, and one robot on a clear indoor route (US-01 through US-05). Existing work includes Python simulation/perception prototypes, a consent/following bridge, and JavaScript screen mocks. The call server, database, production sign-in, and iOS companion are proposed integrations, not deployed services. The caregiver app will request/end calls and display outcomes through I1, use I13 for sign-in, and use I12 for media after resident acceptance; it cannot grant itself CALL or FOLLOW permission.

Evidence boundary. Real public-video experiments did not establish sustained selected-person tracking: the initial acquisition failed, and a later same-sample selection lasted only frames 5–7 before loss. Synthetic RGB-D integration suppressed travel requests but failed stationary hold. Some later bounded simulations improved nominal stopping; feedback-loss cases still failed. No physical stopping, calibrated live person-depth, or integrated mobile call has been demonstrated. Time limits are design targets, not measured deployment guarantees.

2 Data model, the D1 diagram

The call server uses a relational SQLite database. Figure 1 records account-to-robot enrollment and call outcomes. A pairing permits a caregiver's account to request a call; the resident's separate CALL and FOLLOW permissions remain on the robot and can still deny it (US-02, US-03; I1, I2, I14).

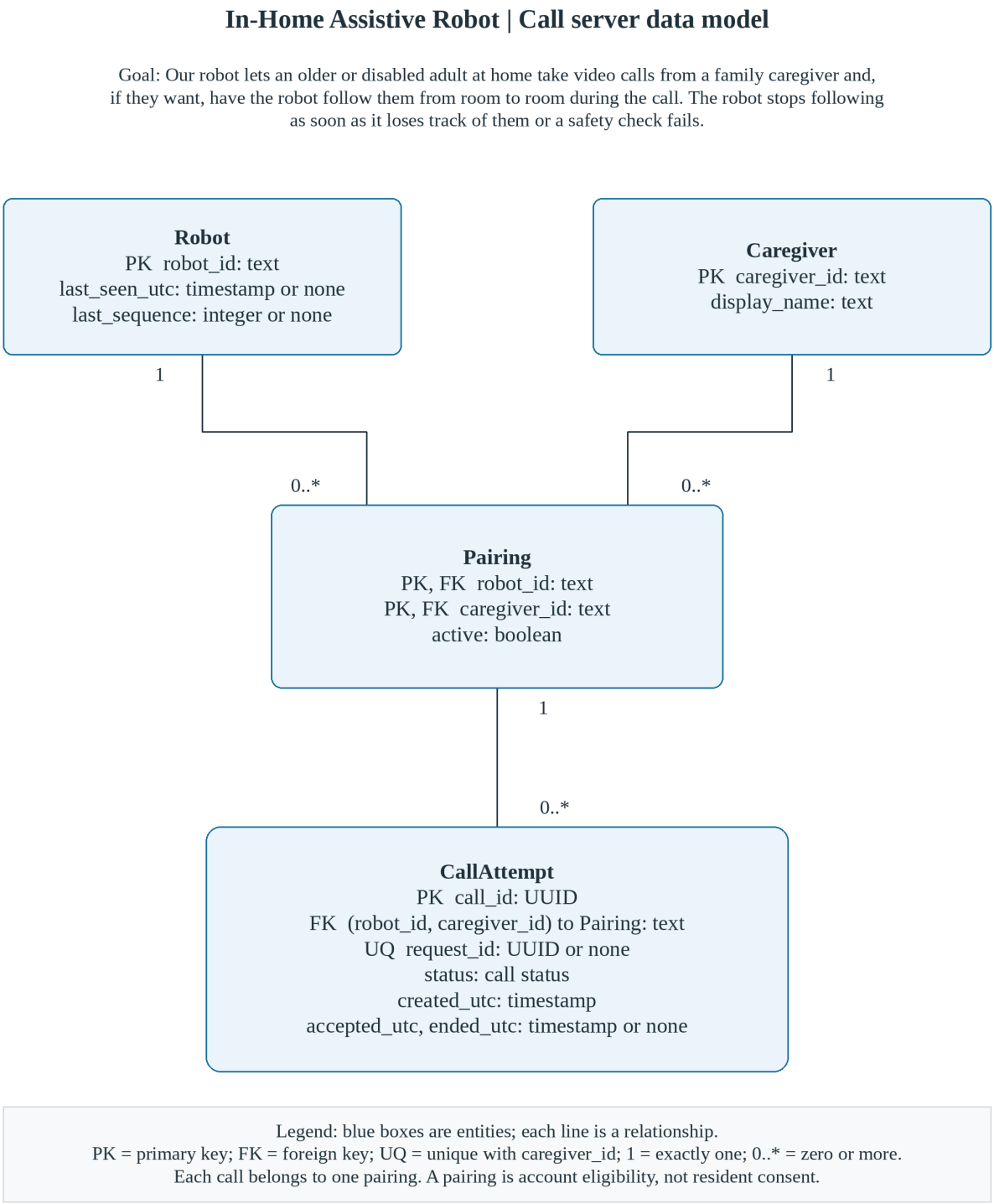


Figure 1. Call server data model

A robot and a caregiver can each have zero or more pairings; each pairing names exactly one of each. One pairing can have zero or more calls, and each call belongs to exactly one pairing through the composite foreign key (robot_id, caregiver_id). An inactive pairing (active = false) stays in the database so its old calls keep their owner. Robot stores last_seen_utc and last_sequence so the server can tell a fresh heartbeat from a repeated one (I2). We store no audio, video, passwords, access tokens, resident locations, or medical records. accepted_utc and ended_utc are null until those events happen, and all timestamps

are RFC 3339 UTC text. The status values are ringing, connected, declined, no_answer, offline, ended, and failed.

Table 1. Structural decisions and requirement links

Decision	Reason
CallAttempt is an entity	Each attempt needs its own time, status, and owner. A single status field on Robot would erase earlier attempts (US-03).
Pairing is an entity	A caregiver and a robot can each be associated with more than one counterpart. The separate pair records that many-to-many relationship and its active/revoked state; it does not replace resident consent (US-02).
Pairing to CallAttempt is one-to-many	One attempt belongs to one robot-caregiver pair; that pair can have many attempts. Sharing one attempt among several pairings would make its outcome ambiguous (US-01, US-03).
Heartbeat sequence is an attribute of Robot	Only the newest heartbeat matters, so we keep one last_sequence value instead of a Heartbeat table that would grow by one row per second. The algorithm compares against it to ignore repeated or out-of-order heartbeats (I2).
Keys and indexes	The Robot and Caregiver primary keys and the composite Pairing key support direct lookups. CallAttempt is indexed on (robot_id, created_utc) so the server can find a robot's latest call quickly. (caregiver_id, request_id) has a unique index so a retried start finds the original call instead of making a second one (I1). A partial unique index on robot_id for ringing or connected calls stops two active calls on one robot (US-03).
Relational store	We chose a relational model in SQLite because the rules above are constraints and transactions, which SQLite enforces for us on a single server. A plain JSON file would make simultaneous updates and uniqueness our problem. Only the call server opens the database file; the robot talks to it over I2.

3 Core algorithms

1. Call attempt and outcome handling

What it is and the problem it solves. The call server takes a caregiver's request and the robot's replies and turns them into one current outcome for each call. If it gets this wrong, the caregiver could see "connected" before the resident has accepted, or a decline could be overwritten by a later message. If it is slow, the caregiver waits on a call that already failed (US-02, US-03; I1, I2, I14).

Inputs and outputs. From the caregiver app (I1): robot_id: text, action: start or end, optional request_id: UUID, call_id: UUID (for end), and a Bearer access token in the request header. From the robot (I2): a heartbeat with sequence: 64-bit nonnegative integer, or an event with call_id: UUID and event: enum. Output: call_id: UUID, status: call-status enum, and last_seen: UTC timestamp or null. A rejected input returns an error code and message, both text. Exact ranges are in Section 5.

How it works. The server checks the token with the sign-in service and then checks for an active pairing. For a new start, it opens one database transaction, looks up the request_id if one was sent, checks the robot's last heartbeat, and inserts one CallAttempt. If the robot is offline the call is recorded as OFFLINE; otherwise it is RINGING and the server sends an invite over I2.

From RINGING, the resident's answer has to come through the robot. DECLINED is final. ACCEPTED records accepted_utc but the status stays RINGING until the robot reports MEDIA_CONNECTED; if that does not come within 2 seconds the call becomes FAILED. With no acceptance after 60 seconds the call becomes NO_ANSWER. The two-second acceptance-to-display target includes network and app time; the server timeout alone does not demonstrate it. A heartbeat only counts if its sequence is greater than last_sequence; if no counted heartbeat arrives for 5 seconds, the robot is reported OFFLINE with its last-seen time. ENDED, DECLINED, NO_ANSWER, OFFLINE, and FAILED are terminal, so reconnecting always means a new call and a new acceptance. Timeouts use the server's monotonic clock, and stored times are UTC for display. On server restart, a transaction closes stored ringing/connected attempts as FAILED with an end time. Every robot then counts as offline until it opens a new authenticated session and sends a heartbeat. A new session retires the old connection and resets its heartbeat sequence baseline, so a robot reboot can start at zero; events from the retired connection are rejected. Calls do not resume automatically.

Complexity at our data size. Each request does a constant number of indexed lookups and one insert or update, so the database work is O(log M) for M stored calls. A planning workload of one robot, one caregiver, and 10 calls a day retained for 30 days gives M = 300; the rate and retention period are assumptions to confirm. At 100 times that history (M = 30,000), log-base-2 M grows from about 9 to 15. These illustrate logarithmic growth, not SQLite page reads, measured comparisons, or latency. Network checks and write contention still need measurement. Heartbeats are the part that grows: 1 write per second for one robot becomes about 100 writes per second for 100 robots. SQLite allows one writer at a time [2], so that is the first thing we would benchmark before scaling up, and the reason PostgreSQL is our fallback in Section 6.

Why this approach. We compared three options. (1) Keeping only the latest status in a file or a Robot column is simpler, but it loses call history (US-03) and cannot settle two starts that arrive together. (2) Letting the robot own the call state and the server only relay messages fails when the robot goes offline: the caregiver would get no answer at all, which is exactly the case US-03 cares about. (3) A server-side state machine in a database transaction, with a partial unique index for one active call per robot, keeps history, handles the offline case, and lets the database reject races. We chose (3). We also push status over a secure WebSocket rather than having the app poll every second, which would add up to a second of delay and constant load for no new information.

Edge cases.

- *Empty or malformed input:* missing fields, bad UUIDs, or unknown fields return 400 INVALID_REQUEST and change nothing (I1).
- *Duplicates:* a repeated start with the same caregiver, request_id, and robot returns the original call. A repeated robot event or heartbeat does not change a terminal call or refresh last-seen time (I2).
- *Ties and races:* if two starts for one robot arrive together, the partial unique index lets one insert

win and the other gets 409 ACTIVE_CALL. If an acceptance arrives after the 60-second deadline, arrival time decides and the call stays NO_ANSWER.

- *Retry ID misuse:* a request_id reused for a different robot returns 409 REQUEST_CONFLICT.
- *Failures:* if the sign-in service or database is down the server returns 503 and never grants access by default (I14). A missing media confirmation or heartbeat never turns into CONNECTED.

2. Selected-person tracking and loss handling

What it is and the problem it solves. This runs on the robot. It keeps track of the one person the resident selected and gives motion control their distance and direction. If it is wrong, the robot can follow a visitor; if it is slow, the robot steers toward where the resident used to be. So it must stop giving a usable position whenever its data is stale, missing, ambiguous, or no longer allowed (US-01, US-02, US-04; I7, I8, I9).

Inputs and outputs. Inputs are the target ID, CALL/FOLLOW gates, permission generation, frame times, detector candidates, an optional selected-instance mask, and a calibrated depth-projection result for that same target, frame, and generation. Pixel positions and projected optical depth are finite floats; timestamps/generation are nonnegative 64-bit integers. Output is an I7 TrackRecord containing target ID, valid/unavailable status, range/bearing or null, capture/expiry times, a reason, coordinate frame, and a false motion-authorization flag. Section 5 defines the types. This is an offline candidate contract; conversion to a qualified robot-frame position remains integration work.

How it works. After accepting a call and pressing Follow, the resident picks one visible person on the screen. On each new frame:

1. *Gate.* Reject the frame if permission is off, its timestamp is in the future or more than 200 ms old, image/depth capture times differ by more than 50 ms, or calibration evidence is missing. The 200 ms freshness gate and 0.8 detection filter come from the existing prototype; the 50 ms skew and association/range thresholds below remain provisional design values.
2. *Associate.* Keep candidates with confidence at least 0.8. For a new explicit selection, use the named candidate without a prior-box match; reject overlapping ambiguous candidates. On later frames, compare intersection over union (IoU) with the last selected box. Accept a best IoU of at least 0.30 only if it beats the second best by more than 0.15. If another kept box overlaps the winner by 0.50 IoU or more, report ambiguity. These geometric tests do not establish identity assurance.
3. *Measure.* Require the exact selected-person mask and a candidate-status result from the calibrated projection adapter, bound to the selected target, frame, and generation. That adapter must check per-zone rays, radial range-error intervals, camera intrinsics/extrinsics, timing/pose uncertainty, and independent clear visibility. The whole projected zone envelope must lie inside the selected mask; a centre pixel or a boolean registration flag is insufficient. Reject inconsistent depth rather than selecting a convenient nearest cluster. Require at least three admitted zones whose entire depth intervals lie in the provisional 0.2–4.0 m domain. For offline inspection, range_m is median RGB optical-axis Z, distinct from raw radial ToF range and robot stopping clearance. Bearing uses arctan((mean projected x - cx_px) / fx_px) in degrees, positive right.

4. *Publish.* Recheck the permission generation and expiry at publication. Emit a VALID offline candidate only while fresh, with motion_authorized=false; otherwise report UNAVAILABLE. Current projection experiments do not grant movement authority.

If any step fails, publish UNAVAILABLE with null range and bearing, latch the loss, and wait for the resident to select and press Follow again. A watchdog publishes the loss even if no new frame arrives. Motion control must reject these motion-disconnected candidates. A future movement path also needs validated identity, a calibrated transform to the robot frame, independent clearance, freshness, resident permission, and qualified stopping. A zero travel request is not a demonstrated physical stop; the recorded feedback-loss failures remain unresolved.

Complexity at our data size. For N candidates and Q already-admitted depth zones, box association is O(N) and a standard-library median sort is O(Q log Q), with O(Q) extra memory [12]. A planning case of 5 candidates and at most 2,268 zones gives $N + Q \log_2 Q$ about 25,300; 100 times the candidates (500) gives about 25,800. These are values of a scaling expression, not counted instructions or measured milliseconds. Camera transport, detector inference, mask construction, full-footprint projection/visibility checks, and control are excluded from this bound and must be budgeted separately. The matched blank-image Mask R-CNN CPU rerun measured about 2.82 s median, excluding the rest of that pipeline; it demonstrates a timing gap rather than person accuracy or a prediction of Pi/Jetson speed.

Why this approach. Nearest-person following could choose a visitor (US-04), so it is rejected. ByteTrack and appearance-based diagnostics have also been explored: raw track IDs continued beyond the selected-target wrapper's loss, while OSNet did not establish sustained selected identity. Neither result permits automatic reacquisition. The proposed baseline therefore retains explicit selection, conservative box association, and a latched loss (US-02, US-04). IoU is an association aid, not proof that two boxes contain the same person. Real identity continuity and masks/depth remain release blockers.

Edge cases.

- *Empty input:* no candidates, no mask, or fewer than three valid depth points gives UNAVAILABLE with reason LOST, NO_MASK, or NO_DEPTH (I7).
- *Duplicates:* a repeated frame_id returns DUPLICATE and does not extend the old expiry; duplicate candidate IDs return INVALID_INPUT.
- *Ties:* two candidates within 0.15 IoU of each other, or overlapping by 0.50 or more, give AMBIGUOUS rather than a guess.
- *Out of order or stale:* an older frame can never replace a newer result; anything older than 200 ms gives STALE.
- *Permission change:* Stop, Privacy, or a new generation during processing means the result is thrown away, so a frame computed before Stop cannot restart following (I8).
- *Known gap:* two people swapping places in the same spot can fool a box tracker. We will test this case directly.

4 Build-versus-reuse decisions

The table separates existing experimental work from proposed integrations. The team-code license is undecided. Library maturity, base license, and intended fit have been reviewed; actual deployment performance, pinned versions, dependency notices, and model/dataset rights still need verification. The resident permission model should reuse the existing consent/following bridge, rather than create a second consent state machine.

Table 2. What the team builds and reuses

Piece	Build or reuse	Library or service	License	Reason (maturity, license, performance, fit)
Call outcome state machine and pairing rules	Build	None	Team code	Build the call-outcome and pairing rules for US-03; integrate the existing resident consent bridge for US-02. The server is not deployed.
Relational storage	Reuse	SQLite	Public domain [1]	Transactions and partial indexes fit one server; load tests, backups, and dependency notices remain necessary [2].
HTTP and WebSocket server	Reuse	FastAPI on Uvicorn	MIT / BSD-3-Clause [3,4]	Maintained, permissive Python stack; async I/O is available, but blocking database/auth work must not block heartbeats.
Request validation	Reuse	Pydantic (used by FastAPI)	MIT [17]	Set extra='forbid', strict scalar types, and finite/range validators; the defaults do not enforce this entire contract [21].
Sign-in and accounts	Reuse	Keycloak	Apache-2.0 [5]	Mature OIDC service candidate; avoids custom passwords. Introspection timing and deployment are unmeasured.
Token verification on the server	Reuse	Authlib	BSD-3-Clause [18]	OIDC/OAuth client candidate; verify asynchronous introspection and error mapping with the pinned release before integration.
Timestamps and UUIDs	Reuse	Python datetime and uuid	PSF [8]	Standard library, so nothing to install; parsing RFC 3339 by hand is a common source of bugs.
Selection, IoU matching, and loss latch	Build	NumPy for array math	Team code; NumPy BSD-3-Clause [19]	Project-specific rules for US-04; NumPy is an optional array tool, not a demonstrated latency guarantee.
Person detector and masks	Reuse	TorchVision Mask R-CNN (first candidate)	BSD-3-Clause code [6]; weight terms checked separately	Existing offline mask adapter; sampled CPU timings exceed 200 ms. SSDLite is a separate box-only baseline and cannot replace selected-instance masks.

Piece	Build or reuse	Library or service	License	Reason (maturity, license, performance, fit)
Calibrated RGB–ToF projection	Build / reuse prototype	Existing standard-library Python contract	Team code; PSF runtime [8]	Reuse synthetic geometry checks; actual calibration, visibility evidence, sensor data, and movement integration are absent.
Simulation	Reuse	MuJoCo	Apache-2.0 [7]	Existing simulation/training tool; bounded successes and failed stopping/latency cases do not establish transfer to hardware.
Caregiver iOS app	Build	Swift and SwiftUI	Apache-2.0 with runtime exception; Apple SDK terms [14,15]	Native iOS UI fits the requested companion; build on Austin's prior SwiftUI work. Real calling, accessibility, and background behavior remain untested.

5 API contract

This section specifies I1 and I2 on the call server and I7/I8 for person tracking. Rules that apply everywhere: JSON field names are case-sensitive and unknown fields are rejected. UUIDs are canonical 36-character strings, and timestamps are RFC 3339 UTC strings such as 2026-10-02T14:30:00Z. Every caregiver request carries its token in an Authorization: Bearer <token> header (1–4096 characters), never in the body, and the server never stores or logs it. Every HTTP error contains an error object with code and message, both text, and no success fields. Map framework validation failures to the specified 400 response; do not leak tokens or raw input in error logs. A pairing makes an account eligible to call; it is not the resident's permission.

Table 3. API summary

Endpoint or method	Inputs	Output	Errors (code: when)
POST /v1/call-commands (I1)	Bearer token; robot_id, action, optional request_id, call_id for end	CallStatus; 202 for a new ringing call, 200 otherwise	400: bad input; 401: bad or expired token; 403: not paired; 404: unknown robot or call; 409: active call or retry ID conflict; 503: sign-in or database down
GET /v1/calls/{call_id} (I1)	Bearer token; call_id in path	CallStatus, 200	400: bad UUID; 401; 403; 404; 503
WSS /v1/calls/{call_id}/updates (I1)	Bearer token on connect; call_id in path	CallStatus on every change	Close 1008: bad or unauthorized token; close 1011: server failure
WSS /v1/robots/{robot_id}/session (I2)	Robot token; heartbeat and event frames	Invite and cancel frames	Error frame INVALID_MESSAGE or STALE_CALL; close 1008: bad robot identity; close 1011: storage failure

Endpoint or method	Inputs	Output	Errors (code: when)
trackSelected (I7, I8)	Frame, candidates, depth, gates, prior state	TrackRecord	TrackError INVALID_INPUT, NO_SELECTION, or DUPLICATE

CallStatus is {call_id: UUID, status: call-status enum, last_seen: UTC timestamp or null}.

Call server: I1

POST /v1/call-commands. The body is a JSON object with these fields:

Input	Type, requirement, and valid values
robot_id	Required text, 1–64 characters, letters, numbers, hyphen, or underscore; an enrolled robot.
action	Required enum: start or end.
request_id	Optional UUID or null for start; when supplied, unique per caregiver. It must be omitted or null for end.
call_id	Required UUID for end; omitted or null for start. It must identify a call for the given robot and authenticated caregiver's pairing. Ending a terminal attempt returns its existing result.

Success, HTTP 202 or 200: a CallStatus object. A new start to an online robot returns 202 with ringing; a start to an offline robot returns 200 with offline. A retry or an end returns 200 with the current or final status. ringing does not mean the resident accepted; only connected does.

Errors: 400 INVALID_REQUEST for invalid types, ranges, or conditional fields; 401 UNAUTHENTICATED for an invalid/expired token; 403 NOT_PAIRED for a valid account without an active pairing; 404 NOT_FOUND for an unknown robot or call; 409 ACTIVE_CALL or REQUEST_CONFLICT for a second active call or a retry ID reused with different input; 503 SERVICE_UNAVAILABLE if the identity service or database is unavailable. The caller may retry a 503, preferably with the same request ID; it must not treat a rejected request as a connected call.

GET /v1/calls/{call_id}. Input is a required UUID path parameter plus the Bearer header. The caller must own the active pairing for the call. HTTP 200 returns a CallStatus. Errors: 400 INVALID_REQUEST for a bad UUID, 401 UNAUTHENTICATED, 403 NOT_PAIRED, 404 NOT_FOUND, and 503 SERVICE_UNAVAILABLE.

WSS /v1/calls/{call_id}/updates. The app opens this socket with the same Bearer header. After the token and pairing checks, the server sends a CallStatus right away and again whenever the status or last-seen time changes; the client sends nothing. The server closes with code 1008 for a missing, invalid, or unauthorized token and 1011 for a sign-in or storage failure. If the socket drops, the app shows that it lost contact and can call GET after reconnecting; a dropped socket never means the call connected.

Example I1 request and response:

POST /v1/call-commands
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{"robot_id": "home-robot-01", "action": "start",
"request_id": "e5c7cb31-343c-4fb8-a632-054e84465d24"}

HTTP/1.1 202 Accepted
Content-Type: application/json

{"call_id": "f34f4e06-d38c-4dbd-a9b1-d11650d06599",
"status": "ringing", "last_seen": "2026-10-02T14:30:00Z"}

HTTP/1.1 409 Conflict (new request_id; another call is active)
{"error": {"code": "ACTIVE_CALL",
"message": "This robot already has an active call."}}

Robot session: I2

WSS /v1/robots/{robot_id}/session. The path has the same robot ID rules as I1. An authenticated robot service token in the Bearer header is required; its subject must match the enrolled robot. Robot-to-server JSON frames are a heartbeat with type:"heartbeat" and integer sequence, sent every second, or an event with type:"event", UUID call_id, and event. Sequence ranges from zero to 2^{63}-1; event is ACCEPTED, DECLINED, MEDIA_CONNECTED, MEDIA_FAILED, or ENDED. Server-to-robot JSON frames are an invite with type:"invite", UUID call_id, text caller_name, and timestamp expires_utc; or a cancel with type:"cancel", UUID call_id, and reason. Reason is ENDED, NO_ANSWER, OFFLINE, or FAILED. Names are 1–80 characters. Sequence is scoped to the current authenticated session: opening a new session retires the old socket and clears last_sequence; subsequent heartbeats must increase it. Old-session messages cannot refresh last_seen_utc or revive a call.

A malformed or invalid event gets {type:"error", code:"INVALID_MESSAGE" or "STALE_CALL"}; it cannot change a terminal call. The server closes with WebSocket code 1008 for a bad robot identity or repeated invalid messages and 1011 for an internal storage failure. When the session drops, the server marks the robot offline after 5 seconds, and the robot ends any call in progress (D0, I2).

Person tracking method: I7 and I8

trackSelected(input) -> TrackRecord or TrackError. This is an in-process function call on the robot, not an HTTP endpoint. Every field below is required unless marked optional. Time values are integer milliseconds in one robot monotonic clock domain, from 0 to 2^{63}-1.

Input	Type, requirement, and valid values
Selection and gates	target_id: text, 1–64 characters; call_accepted and follow_allowed: booleans; generation: nonnegative integer.
Time and frame	now_ms, frame_id, capture_ms: nonnegative integers. Image width and height: positive integers, each at most 1920, with width times height at most 2,073,600 pixels.
Candidates	Array of 0–500 records: candidate_id is unique text of 1–64 characters; box is [left, top, right, bottom] in integer pixel coordinates with positive area; left/top are inclusive, right/bottom exclusive, all within image bounds. confidence is a finite number in [0,1]. mask is an optional image-sized boolean array.
Depth projection	Required record: target_id, candidate_id (text, 1–64 characters), frame_id, generation, and capture_ms (nonnegative integers); these must match the selected observation. status: candidate/not_observed/uncertain; source: synthetic/offline/hardware. points: 0–2,268 already-admitted records with finite pixel x, y, positive finite optical_z_m, and z_interval_m: two ordered finite positive bounds enclosing that depth.
Calibration	fx_px: finite number greater than zero and at most 100,000; cx_px: finite number from zero inclusive to width exclusive, in pixels for the rectified RGB image. calibration_id: text, 1–64 characters, naming the validated revision for this image mode. The upstream adapter also needs measured zone rays, intrinsics/extrinsics, bounded reprojection/range error, synchronized capture times, and per-zone clear visibility; its validated calibration revision must match the frame mode. Nominal field of view is insufficient.
Selection state	Optional selected_candidate_id: text or null, required for a new explicit selection. Optional prior: null or a record with generation, last_frame_id (nonnegative integers), last_box (four in-bounds pixel integers), and loss_latched (boolean).

New selection requires a fresh resident Follow action, a selected candidate, and either no prior state or a generation greater than the prior generation. A latched loss cannot be cleared by later frames alone. The tracker atomically updates its per-target prior state when processing a frame; the next call receives that state. Duplicate frames do not update it.

TrackRecord: fields are target_id: text, tracking_status: valid/unavailable, range_m and bearing_deg: finite number or null, capture_ms and expires_ms: integer milliseconds, reason: enum, range_frame: constant rgb_optical_z, and motion_authorized: constant false. Range is median RGB optical-axis Z in metres; bearing is in degrees, positive right. It is not a robot-frame separation or clearance. Valid means an offline candidate with reason OK, expiring by capture + 200 ms; it never grants movement. Unavailable has null range/bearing and reason CONSENT_OFF, STALE, AMBIGUOUS, LOST, NO_MASK, NO_DEPTH, or BAD_CALIBRATION. Missing calibration gives BAD_CALIBRATION; not_observed, uncertain, or missing visibility gives NO_DEPTH. Stop or a generation change invalidates earlier output immediately. A qualified movement interface requires separate base-frame conversion, identity/clearance checks, and physical validation in D2; it cannot reinterpret this offline range as a robot-frame command.

TrackError: {code:enum, message:text}. INVALID_INPUT covers missing fields, invalid bounds or

duplicate IDs; NO_SELECTION covers a missing required resident selection; DUPLICATE covers an already processed frame. INVALID_INPUT and NO_SELECTION make the caller request a stop. DUPLICATE must not extend the prior record's expiry; the watchdog requests a stop when freshness expires, even if no new method call arrives. The proposed integration serializes permission-generation checks with publication; this atomic cross-component behavior still needs testing. Offline bridge tests alone do not establish it on hardware.

Versioning. The HTTP and WebSocket APIs are versioned by the /v1 path prefix and the tracking method by a version constant in its module; removing or renaming a field, changing a type, unit, required input, status meaning, or error behavior is a breaking change that requires /v2, while adding an optional output field is not breaking (adding an input field is, because v1 rejects unknown inputs).

6 Technology choices with justification

Each paragraph covers team skill fit, licensing, community support, performance, and cost/hosting. These are proposed selections for review; unmeasured capacity, prices, and hosting allowances are not established facts.

Database: SQLite. *Skill:* Python is already used for simulation and perception; the team will still need to implement the SQL schema and migrations. *License:* SQLite's core is public domain, with dependency and deployment notices checked separately [1]. *Community:* official documentation covers transactions, indexing, and deployment [2]. *Performance:* one server is a reasonable starting design, but neither one-robot latency nor 100-robot write contention has been benchmarked. *Cost and hosting:* no database license fee, but durable storage, backups, and administration require an approved host. *Alternative:* PostgreSQL adds an independently operated database service and is the candidate if measured write contention or multi-host deployment warrants it.

Backend: Python with FastAPI on Uvicorn. *Skill:* existing Python experiments provide relevant experience; JavaScript screen mocks are a separate existing code path. *License:* Python PSF, FastAPI MIT, and Uvicorn BSD-3-Clause [3,4,8]. *Community:* both server projects have maintained official documentation. *Performance:* asynchronous I/O can overlap network waits, provided database and authentication work do not block the event loop; a worker count or two-second response guarantee cannot be chosen without measurements. *Cost and hosting:* the software has no license fee; compute, monitoring, and integration still cost resources. *Alternative:* Node.js/Express would use the team's JavaScript experience, but Python keeps the initial server aligned with the simulation/perception tooling.

Caregiver front end: iOS app in Swift and SwiftUI. *Skill:* Austin's prior Swift/SwiftUI app work supports the choice, with Gavin proposed to design the screens. The robot companion itself is not implemented. *License:* Swift is Apache-2.0 with a runtime exception; Apple SDK and team-code terms are separate [14,15]. *Community:* Apple documentation and Swift forums support development; VoiceOver and text sizing still need app-specific tests [16]. *Performance:* foreground call requests and status updates are the initial target. Incoming background calls would need a separately designed APNs/PushKit/CallKit path; no locked-phone ringing or hardware-encoder result has been demonstrated [20]. *Cost and hosting:* a free developer account supports own-device testing; wider distribution may need the $99/year

program, plus a Mac/iPhone and backend hosting [15]. *Alternative:* a browser companion remains a cross-platform option; the requested iOS direction motivates native development, not an untested claim that it solves notification reliability.

Identity and messaging: Keycloak, HTTPS, and secure WebSocket. *Skill:* Python integration is plausible, but production Keycloak operation is not recorded; Austin is the proposed integration owner. *License:* Keycloak Apache-2.0, with Authlib BSD-3-Clause as a client candidate [5,18]. *Community:* official OIDC documentation supports integration [9]. *Performance:* token introspection adds a network operation; its delay, resource footprint, and effect on heartbeats are unmeasured. *Cost and hosting:* no software license fee, but identity storage and operations may need a separate service or a larger host; resource sizing remains to be measured. *Alternative:* managed identity such as Firebase Authentication could reduce administration, but introduces provider terms and costs to evaluate; no provider is deployed.

On-robot perception and hardware: Python/TorchVision, host to be selected. *Skill:* existing offline detector and projection work supports this software direction. *License:* TorchVision/PyTorch base code and bundled dependencies have separate notices; dataset/weight rights remain to be resolved [6,13]. *Community:* official TorchVision, ST, and Raspberry Pi documentation provide supported integration references [6,10,22]. *Performance:* SSDLite is box-only; the measured Mask R-CNN CPU baseline misses 200 ms. Neither Pi 5 nor Jetson performance has been measured. The purchased/shipped VL53L9CXV0VE/1 is a bare depth component, not an RGB-D USB device; carrier, host, real frames, calibration, and power integration remain unverified. A Pi 5 candidate has two shared camera/display ports, so RGB CSI + ToF MIPI + DSI display needs a different verified route [10]. *Cost and hosting:* board variant, carrier, camera, display, storage, power and integration prices need quotes; no replacement build budget is approved. *Alternative:* a Jetson is an evaluation candidate if compute needs justify it, without a promised speedup or automatic hardware switch.

Hosting: proposed Ubuntu Server VM. *Skill:* public-server operating experience is not documented; Austin is the proposed deployment owner. *License:* Ubuntu comprises packages with distinct licenses [11]. *Community:* official platform and server-project deployment documentation are available. *Performance:* test the complete server, identity service, and any media relay against heartbeat and call targets before choosing RAM or worker counts. *Cost and hosting:* provider, VM size, persistent disk, backups, TLS operation, relay traffic, and monthly allowance remain unapproved. *Alternative:* a laptop can support local integration, but remote reachability and availability require separate validation.

Source notes

[1] SQLite copyright and public-domain core; [2] SQLite deployment and single-writer guidance; [3] FastAPI project and license; [4] Uvicorn project and license; [5] Keycloak project and license; [6] TorchVision project, code license, and weight notice; [7] MuJoCo project and license; [8] Python license; [9] Keycloak OIDC endpoints; [10] Raspberry Pi connector documentation; [11] Ubuntu package and license policy; [12] Python standard-library median; [13] PyTorch code and bundled dependency licenses; [14] Swift license; [15] Apple developer tools, testing, and distribution; [16] SwiftUI accessibility fundamentals; [17] Pydantic project and license; [18] Authlib project and license; [19] NumPy license; [20] Apple incoming VoIP notification requirements; [21] Pydantic validation configuration; [22] ST VL53L9CX component specifications.

In-Home Assistive Robot

Detailed System Design

Design D2

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

November 11, 2026

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI assisted with drafting and LaTeX formatting.

Peer feedback recorded; follow-up validation pending.

1 Header, conventions, and D2 overview

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

The system is a compact robot built by the team, a caregiver's iOS companion, and a call-signaling service. The resident accepts each call, chooses whether to follow, and can revoke access locally. D1 detailed the call server's relational model and selected-person tracking. D2 completes the resident controls, call manager, media, motion, physical platform, and caregiver app, adds authentication, notifications, configuration, and logging, and closes the integration boundaries of the two D1 components.

The first complete user journey remains an invited call and a slow, requested transition on one clear indoor floor. D2 defines intended behavior for that journey; it does not certify a working household robot. Broader reminder, delivery, home-control, and help features remain outside this design's requirement baseline.

Conventions. In both figures, solid rectangles are software states or decisions; the labels identify which. Dashed rectangles are failure or revocation outcomes. Arrow labels state the event or guard for a transition; arrows carry control, not physical force. State held during a session is volatile unless a component explicitly names a persistent record. The data model uses PK for primary key, FK for foreign key, and 1 / 0..* for exactly one / zero or more; the D1 entity diagram remains the model for the call database. Each figure repeats its own legend, project title, and goal.

Known gaps and open questions. The team is exploring both a statically supported base with two driven wheels and a rear caster, and a two-wheel balancing base; neither is selected as the final platform. Both use the interfaces below, with different stop responses. Unrestricted household operation is outside the first prototype: moving trials require a supervised, bounded dry-floor area, verified privacy guards and the appropriate physical support. Hardware dimensions, motor/driver parts, stopping measurements, camera/depth integration and hosting costs remain to be confirmed. Until a matching qualification record exists, actuation stays disabled. The permission, expiry, fault and recovery rules are design decisions; their hardware performance is not a completed test result. Clare Hooper's October 26 review and our single outgoing issue are recorded in Section 6, with proposed November 18 follow-up targets.

The newer stationary prototype has delivered generated video/audio in eight same-machine calls. It uses Node, HTTP/SSE, browser WebRTC, and fixture identities; it is not the proposed FastAPI, Keycloak, and SwiftUI deployment, a LAN test, or a physical camera trial. New SAM-mask decoding and freshness work is also offline and cannot authorize motion. Its 200 ms age limit is a development bound; the measured stopping envelope must cover the entire loss-to-braking delay. These experiments guide the adapters below without replacing their missing real-world evidence.

Ownership. Gavin leads resident interaction, screen layout, caregiver workflows, and accessibility evaluation. Liam leads 3D modeling, chassis/mount design, fabrication, assembly, fixtures, power/mechanical integration, and physical movement/stopping tests; Austin implements software, sensing, communication, and control with Liam. This preserves the D0 ownership boundaries.

2 Finished components, the D2 diagrams

Shared records and execution rules

A UUID is a canonical UUID string. An ID is 1–64 ASCII letters, digits, underscores, or hyphens. A uint is an integer from zero through 9,223,372,036,854,775,807; booleans are not integers at the ingestion boundary. Floats must be finite. ms means integer milliseconds in one local monotonic clock; UTC is used only for durable display/audit times. Time from a remote computer never replaces the robot's local capture time.

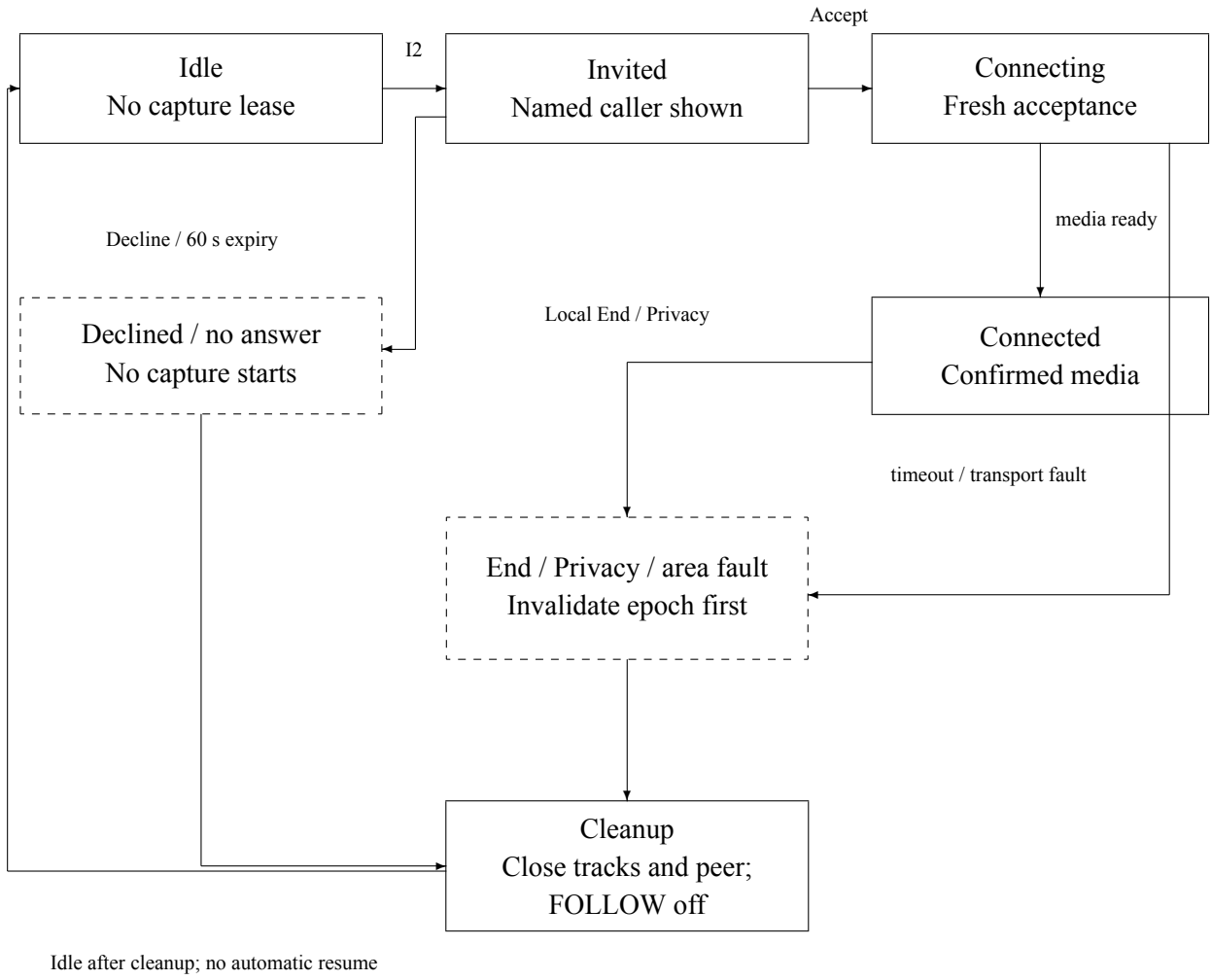
Context is {boot:UUID, call:UUID|null, epoch:uint, sequence:uint}. Call is null only while idle; signaling/media require a nonnull call. Reboot changes boot; End, Privacy, caller revocation, and authority failure invalidate the session epoch. Every asynchronous completion checks current context before publishing or affecting capture. A late completion from an old call neither revives that call nor tears down a newer one.

Result is either {ok:true, value:T} or {ok:false, code:Code, message:text}. Text is 1–160 characters with no credentials or raw media. Codes are INVALID_INPUT, DENIED, STALE, BUSY, UNAVAILABLE, TIMEOUT, FAULT, or UNQUALIFIED. The contracts below assign when they occur. Existing D1 HTTP codes remain unchanged. Live queues are bounded; revocation takes priority over ordinary requests. A separate selection_epoch:uint changes on target loss, reselection, or FOLLOW revocation; it does not end the call or change its media epoch.

AuthoritySnapshot is {context:Context, call_state:CallState, media_allowed:bool, follow_allowed:bool, selected_target:ID|null, environment_revision:uint}. Both permissions start false. This revision identifies the current EnvironmentState; leases bind to it, so an area-policy change invalidates old work. **CallLease** adds approved audio:bool, video:bool, a local expires_ms:uint, and environment_revision:uint; it is valid only for the current accepted call and allowed area revision. The application never treats a transport acknowledgment as resident acceptance or observed capture. TrackingState is status: unselected/candidate/lost/unavailable, selection_epoch:uint, target:ID|null, reason:Code|null; capture state is context:Context, camera:on/off/unknown, microphone:on/off/unknown.

In-Home Assistive Robot | Resident-controlled call lifecycle

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.



Legend: solid boxes = call states/actions; dashed boxes = denied or revoked outcomes. Arrows = guarded events. I2 is signaling, not media. Area/Privacy faults revoke media and tracking; a new call requires new acceptance.

Figure 1. Call lifecycle and error paths

Resident controls

Data. No persistent data. ResidentView holds caller name (1–80 characters), CallState, following/stopping/unknown, privacy on/off, and separately observed camera/mic on/off/unknown. An action carries the current Context and an increasing sequence. Camera icons reflect observed capture, not merely the requested gate.

Behavior. Show the named invitation and accessible sound; accept/decline is explicit. Follow requires an accepted call and a selected target. Stop following withdraws FOLLOW without ending the call. End withdraws both scopes; full Privacy also requests all application capture off. Unknown sensor state is shown as unknown. A privacy exit restores no call, media, or following. Buttons must be reachable from

a seated position; a separate switch fixture is acceptable if the low screen is not reachable. In response to PR-04, use a small, consistent set of plainly labeled actions, readable high-contrast text, and status conveyed by words as well as color. The resident workflow requires neither a smartphone nor a remembered voice phrase. Gavin will walk through accepting/declining a call, stopping, enabling Privacy, and recognizing capture state with varied text size, reach, hearing, vision, dexterity, and technology familiarity; record task errors and assistance instead of claiming usability for everyone. Physical and participant evaluation remain pending.

Interface. I3 act(context, action, target:ID|null) -> Result<AuthoritySnapshot>; action is accept, decline, end, follow, stop, or privacy. Target is required only for Follow. Invalid or old context returns STALE; Follow without acceptance/selection returns DENIED. I15 renders ResidentView; unsupported action returns INVALID_INPUT. I4 is a supervised wired input bundle: asserted/open Stop withdraws movement; asserted/open Privacy withdraws all RGB/microphone capture and movement. Guard and operator-enable inputs have the separate scope defined below. Contact debounce must never delay assertion; release alone cannot rearm. Physical wiring is unqualified. Handler/owner: resident controls, Gavin; safety input implementation, Liam/Austin.

Call manager and local authority

Data. No persistent permission grants. State is Idle, Invited, Connecting, Connected, or Closing, plus the current Context, CallLease, selected target, and a separate FOLLOW grant. Retain the existing consent/bridge rules as the behavioral reference; do not build a second writable consent authority.

Behavior. Validate an authenticated, paired invitation; arbitrate one active call. Accept grants only that call's requested media. Commit a revocation/epoch change before initiating cleanup, then revoke media, delegated tracking capture and motion at their owners. End/session revocation also drops the shared capture-enable latch; only tracking loss can leave the media lease intact. End, full Privacy, caller revocation, session loss, or expiry invalidate pending work. A watchdog expires leases even if no new callback arrives. Admission also requires a current allowed EnvironmentState. An excluded or unknown operating area revokes FOLLOW, the media lease and tracking capture; cleanup follows the same epoch-first Privacy path. An operator-enable release stops movement alone when the area and privacy guards remain valid. No caregiver override bypasses that boundary (PR-02/03). Keep an active call for ordinary tracking loss, Stop following or an operator-enable release when media permission remains valid; end it when the signaling session itself is lost, matching D0 I2. Call setup has a two-second acceptance-to-visible-connection target, measured across the full path, not just a server timer (Figure 1).

Interface. I5 setMedia(lease:CallLease|null) -> Result<MediaState>; I6 setFollow(context, selection_epoch:uint, enabled:bool) -> Result<MotionState>; I8 select(context, selection_epoch:uint, target:ID|null, enabled:bool) -> Result<TrackingState>. Null/false revokes. Old context returns STALE; competing invitation BUSY; absent authority UNAVAILABLE. An unresponsive service cannot block permission revocation. Austin owns this component.

Operating-area admission within the local authority. EnvironmentPolicy is a persistent, versioned local record: {revision:ID, area:ID, barriers:ID, view_check:ID, permit_ms:uint[1..300000]}. The default permit is 300,000 ms for one supervised trial. Barrier/view IDs refer to a documented route and camera-

view check for the current geometry, not an inferred room label. The checked route has closed physical barriers to bathrooms, wet areas and stairs; an opaque screen or closed door excludes bathroom views over the full permitted route, camera field of view and turns. Without those physical conditions, no area is approved. Final room recognition or autonomous geofencing is not assumed.

EnvironmentState is volatile: {boot:UUID, policy:ID, area:ID|null, revision:uint, status:allowed/excluded/unknown, expires_ms:uint, guard:closed/open/fault, operator_enable:bool}. It starts unknown on boot, relocation or configuration change. A local operator inspects the dry route, barriers, full camera view and support fixture, then explicitly arms the trial on the robot's commissioning panel. This physical operator action is separate from a network maintainer/caregiver role. A supervised, normally closed guard circuit reports barrier/privacy-guard state; a separate hold-to-enable input is required for movement. Neither is a caregiver command. Initial moving trials therefore require an operator; unattended household use is not claimed.

Admission steps. (1) Reject an old boot/policy, incomplete check or unqualified profile. (2) With the guard closed, store a new revision and local expiry; publish allowed. (3) Each control tick requires that revision, an unexpired permit and current guard state; movement also requires operator enable. (4) At now>=expires, a guard opening/fault or relocation, increment the environment revision and call epoch before asynchronous cleanup, revoke all RGB/microphone/tracking leases and request a latched stop. The wired guard also drops the capture-enable latch independently of application queues. Depth/encoder inputs needed for stopping are separate. (5) Closing a guard or returning power cannot rearm: a fresh local area check, new call acceptance and explicit Follow/selection are required as applicable.

Local contract. armArea(policy:ID, area:ID, checks:AreaChecks, expected_boot:UUID) -> Result<EnvironmentState>; AreaChecks has dry_route, barriers_closed, full_view_screened and support_ready, all booleans that must be true and match the manifest/physical inputs. False checks give DENIED; wrong boot/revision STALE; absent guard UNAVAILABLE; missing qualification UNQUALIFIED. Only the local commissioning panel may call it. invalidateArea(reason) is idempotent, returns the new EnvironmentState, and accepts guard_open, guard_fault, moved, expired, privacy or config_changed; unknown reason gives INVALID_INPUT. The authority reads a fresh state directly each tick; remote messages cannot publish allowed. Austin owns this logic; Liam owns the guard/support fixtures; Gavin owns the local check and status wording.

Audio/video calling

Data. No persistent media. One volatile peer connection, accepted lease, bounded SDP/ICE queues, and capture-state observations. MediaState is off, connecting, connected, closing, failed, or unknown. Device buffers are never written as a call recording.

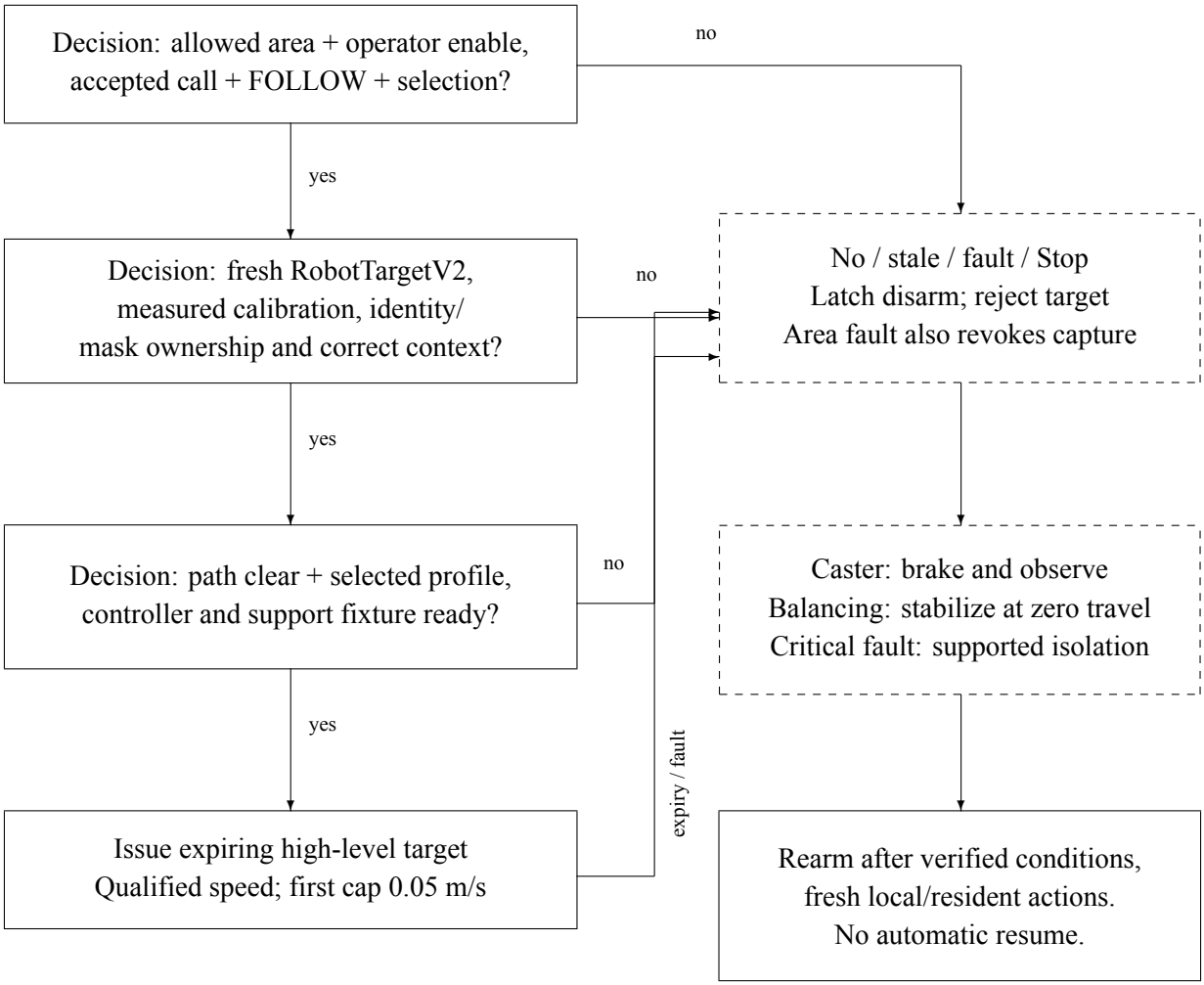
Behavior. Open devices only after lease admission. Check context before attaching tracks and on every asynchronous completion. On revoke, detach/stop outgoing tracks, close the peer, clear queued signaling and volatile media buffers, then release its platform capture lease; a failed physical close reports unknown, never camera-off by assumption. A provisional two-second negotiation deadline produces FAILED, not an invented connection. No automatic redial or ICE restart. A new call starts a fresh peer/lease. Physical device privacy still requires fixture verification. PR-03 excludes bathroom camera use, including a view

through an open doorway: approved placement must keep the bathroom out of view, not merely keep the wheels outside. No ambient bathroom audio monitoring is added. Full Privacy also cancels tracking, clears its image/mask buffers and drops the platform capture-enable latch; ending media alone must not leave the tracking camera open. If the allowed area or capture state is uncertain, deny capture, show unknown where appropriate, and require local recovery; an off icon is not evidence of device shutdown.

Interface. I10 capture(lease) -> Result<CaptureState> and close(context) -> Result<CaptureState>; failures are DENIED, STALE, UNAVAILABLE, or FAULT. I12 uses encrypted WebRTC media with a configured direct/relay route [1]. Signaling is WSS /v2/calls/{call}/signal, authenticated as a participant. Each frame has Context, type offer/answer/ice/close, and body. SDP is text at most 65,536 UTF-8 bytes; ICE is {candidate:text, mid:text|null, mline:uint|null} at most 4,096 bytes. Limit 128 pending candidates; drain only after the matching remote description. Overflow/invalid messages close 1008, internal failure 1011, timeout ends the attempt. Never log SDP, ICE addresses, or tokens. The adapter is new; the local fixture's loopback ICE rewriting must not be used on a LAN. Austin owns media.

In-Home Assistive Robot | Following admission and failure handling

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.



Each tick rechecks expiry, local Stop/Privacy and platform feedback; excluded/unknown area revokes access; no remote override.

Legend: solid boxes = decisions/actions; dashed boxes = failure states. Arrows = guarded control transitions. A tracking-only stop can retain the call. A zero travel request is not a validated physical stop; the profile selects the fault response.

Figure 2. Movement guards, watchdogs, and error paths

Person tracking: D1 completion and adapter boundary

Data. No persistent images or target identities. Keep a bounded capture ledger (128 entries), boot/selection epoch, original acquisition times, selected instance, calibration revision, and decoded masks in memory. D1's optical-Z TrackRecord remains an offline inspection type with motion_ authorized=false.

Behavior. The desired SAM path is an optional mask provider, not a motion authority. Bind every reply to exact provider/version, request, capture, and selection. Enforce resource limits before mask decoding; accept only verified full-frame placement, not guessed resize/rotation. Reject unknown/duplicate frames, expired/old selections, empty or ambiguous masks, missing calibrated depth, and late completions. An empty provider result means no observation. SAM object IDs are not cross-request personal identity [2]. Current provider probes and normalized-mask adapters do not establish live tracking.

Interface. I9 supplies sensor observations; the private mask adapter exposes admit(capture, selection, decodedMask) -> Result<OfflineCandidate>. Its output cannot issue movement commands. I7 gains a separate RobotTargetV2: {context:Context, selection_epoch:uint, frame:uint, target:ID, capture_ms:uint, expires_ms:uint, forward_m:float, right_m:float, uncertainty_m:float, calibration:ID}. Distances are in the robot's calibrated level-ground frame, forward/right positive, uncertainty nonnegative. A rigid camera-to-base transform plus measured support/pose is required; radial ToF distance or D1 optical Z cannot be renamed forward distance. There is currently no qualified producer of this type: emit UNQUALIFIED, not fabricated coordinates. Loss invalidates the target and requires explicit reselection. Austin owns tracking; Liam supplies measured sensor mounting/pose data.

Motion control

Data. No persistent motion commands. MotionState is disarmed, following, stop_requested, settled, fault or unknown. Keep the target lease, local sequence, environment revision, stop anchor and fresh platform feedback in memory. A persistent StopEnvelope belongs to a measured geometry/profile revision: maximum allowed speed, worst loss-to-braking delay, measured braking-distance bound as a function of speed, feedback-error margin and evidence ID. Missing evidence keeps the motion feature disabled.

Following algorithm. Figure 2 is the admission flow. (1) Require the current accepted call, resident FOLLOW/selection, allowed area, held operator enable, fresh calibrated RobotTargetV2, independent clearance and a qualified platform profile. (2) In the robot frame compute distance sqrt(forward*forward+right*right) in metres and right-positive heading atan2(right,forward). Use a 0.9 m approach setpoint; request min(speed_cap,max(0,0.4*(distance-0.9))) m/s. Admit tracking only with uncertainty at most 0.05 m; when distance plus its uncertainty is at most 1.0 m, latch an arrival stop with both travel and turn zero. This gives a finite arrival threshold inside the one-metre requirement rather than asymptotically approaching it. Before arrival, suppress forward travel when heading error exceeds 30 degrees. Request heading correction of one degree/s per degree of error, limited to -30..30 degrees/s. These gains are initial test values. (3) Clamp valid computed requests to the qualified envelope; reject malformed/nonfinite external inputs. Publish only the newest DriveCommand with expiry min(target.expires_ms,now+100); reusing an old target cannot extend its acquisition deadline. Each tick repeats admission; a lost target latches stop and requires explicit reselection. Arrival retains the call; another movement needs a new resident Follow action.

The absolute speed limit remains 0.2 m/s. The initial candidate cap is 0.05 m/s and can be enabled only after the envelope verifies v*t_loss+d_brake(v)+margin<=0.025 metres. Here t_loss includes sensor age, loss detection, scheduling, transport, clock uncertainty and command expiry, not just the controller

period. This resolves the conflict between a 200 ms observation-age allowance and a 25 mm stopping target at higher speed. Record peak excursion and the final stop separately. Reject the motion profile if no measured speed satisfies the bound. No stopping distance is claimed from the formula alone.

Stop and fault steps. Stop following, target loss, operator-enable release or command expiry first latches disarm and prevents fresh travel commands. In the caster profile, command zero wheel speed and the driver's documented brake mode; once settled, retain the brake/hold state. In the balancing profile, request zero travel/turn while the local balance controller remains active; a normal Stop never blindly cuts its balancing torque. The balancing exploration is permitted only in a passive support fixture that arrests a fall/runaway; free-floor balancing remains disabled. Both branches use a bounded test area and physical restraint for power/driver/feedback fault-injection tests. On a critical feedback/controller/driver fault, the independent safety path disables drive, the fixture provides physical containment, and motion is fault/unknown until inspected. For the caster candidate, unpowered rolling/holding must also be measured; dynamic braking is not assumed to work without power. Neither profile is released for unrestricted movement without a verified physical fault response.

Report settled only from fresh calibrated feedback: final displacement at most 25 mm from the loss anchor and speed below 5 mm/s for a full second within 10 seconds. If it does not settle by then, or feedback disappears, latch fault/unknown; do not report a software stop request as physical success. Ordinary tracking/Follow stops can retain the call; area/Privacy faults revoke all capture. Rearm requires inspected healthy conditions, a fresh local enable and explicit resident Follow/selection; fault reset alone never starts movement.

Interface. I11 command(request:DriveCommand) -> Result<MotionObservation>. DriveCommand is {context:Context, selection_epoch:uint, sequence:uint, environment_revision:uint, v_m_s:float, turn_deg_s:float, expires_ms:uint}; v is 0–the qualified cap (never over 0.2), turn is -30..30 degrees/s and the host deadline is at most 100 ms ahead. I6 returns MotionState; I7 consumes RobotTargetV2. Add stop(context, reason) -> Result<MotionObservation> with reason resident, target_lost, expired, area, feedback or power; it is idempotent and never grants motion. Invalid input gives INVALID_INPUT; old boot/epoch/revision/sequence or expired request STALE; closed permission DENIED; missing envelope/profile UNQUALIFIED; device/feedback failure FAULT. Liam owns movement/fixture testing; Austin owns the supervisor and adapters.

Robot platform

Data. PlatformManifest is persistent: revision:ID, geometry:ID, profile:caster/balancing, controller:ID, calibration:ID, wheel_radius_m:float, track_width_m:float, driver:ID, stop_envelope:ID|null and qualified:bool (default false). Radius/width must be finite and positive. The caster candidate has two driven wheels and a rear caster; the balancing candidate has two driven wheels and a local stabilizer. A profile change invalidates the manifest, stop envelope and area approval; simulation success for one never qualifies the other.

MotionObservation is volatile: Context, controller_boot:UUID, sequence:uint, sample_ms:uint, speed_m_s:float, turn_deg_s:float, state:MotionState, path_clear:bool|null, support_ready:bool|null, guard:closed/open/fault and fault:Code|null. Null means unknown, never clear/ready. No motion com-

mand survives reboot. SensorObservation has frame:uint, capture_ms:uint, rgb_handle:ID|null, depth_handle:ID|null, rgb_ms:uint|null, depth_ms:uint|null, depth_status:available/unavailable, calibration:ID and driver_status:available/fault. Capture_ms is the oldest acquisition time in the matched pair. An unavailable buffer has a null handle/time; an uncalibrated or unsynchronized pair cannot produce a target. RGB/depth handles refer to bounded process-local buffers using the D1 image/point limits and units; the calibrated depth record carries the D1 optical-depth points and their bounds. Handles belong to the current boot/frame and expire with that capture. RGB access requires the current media or delegated tracking lease; full Privacy invalidates those handles and clears their buffers. Independent depth data used only by the stop controller remains available under its separate local safety scope. CaptureState reports camera/microphone on/off/unknown separately.

Controller behavior. The proposed common boundary is a dedicated local controller over USB serial, not a network motor endpoint. The caster adapter uses right-positive turn: with omega in radians/s and track width b, left/right linear wheel setpoints are v+omega*b/2 and v-omega*b/2; divide by wheel radius for rad/s and apply the measured motor polarity; a local PI feedback loop regulates each wheel with bounded output, anti-windup and integral reset when disarmed. Gains/output limits are calibration data tied to the manifest and require bench tuning. The balancing adapter passes body travel/turn requests to its experimental local stabilizer and requires current IMU/wheel feedback plus the engaged support fixture. Its stabilization policy/gains remain an experimental asset, not evidence of floor readiness. The controller runs a provisional 10 ms supervisory tick, reports feedback every 20 ms and enters the profile's stop response on a 100 ms command expiry; feedback older than 60 ms is a fault. All periods require load testing. Local Stop/guard contacts and an independent watchdog override queued commands. On critical watchdog/feedback failure drive is isolated in the supported test configuration; after power loss there is no claim of fresh telemetry or a confirmed stop.

Hardware and capture behavior. Liam owns 3D modeling, chassis/mounts, fabrication, mass/stability, wiring strain relief, guards and support fixtures for both candidates. Keep bathrooms outside both the route and camera view. Gavin/Liam compare full-scale visibility/reach mockups, protrusions, approach clearance and parking outside walking routes/doorways. The bare depth component still needs carrier, power/firmware, host, RGB camera and measured calibration. One platform capture owner brokers RGB/microphone access for media and tracking; consumers cannot open a second independent camera. The normally-off capture-enable latch drops on End/session revocation, full Privacy, area-guard fault, authority failure or power loss, irrespective of pending callbacks. Driver closure and gate feedback must both confirm off; otherwise show unknown. Clear volatile image/audio/mask buffers on revoke. Stop-related depth/encoder inputs stay separate, so removing call capture does not remove required stopping feedback. Releasing a contact cannot restart capture.

Interfaces. I9 readSensors() -> Result<SensorObservation>; I10 capture(lease:CallLease) -> Result<CaptureState> and close(context) -> Result<CaptureState>; I11 takes DriveCommand/stop above. No lease gives DENIED; missing device UNAVAILABLE; invalid schema INVALID_INPUT; wrong calibration/profile UNQUALIFIED; driver or capture-gate failure FAULT. The wired I4 status adds guard and operator-enable inputs beside Stop/Privacy; open/fault is restrictive. I9 also provides resolveBuffer(handle:ID, context:Context) -> Result<FrameBuffer>: RGB is uint8[height,width,3] with each dimension <=1920 and at most 2,073,600 pixels; depth is the D1 bounded optical-depth point record, not raw radial range. Missing/expired handles give STALE, absent permission DENIED

and unqualified projection/calibration UNQUALIFIED. Buffers stay in the host adapter; they are not motor-channel payloads. The controller channel is bounded, line-delimited UTF-8 JSON (maximum 4,096 bytes/frame) with type command/stop/status/sync, controller boot, increasing sequence and the typed fields above. A sync reply echoes sequence plus sample_ms:uint and controller_boot:UUID; H1/H2 are measured only by the host. Channel replies use Result; invalid frame INVALID_INPUT, old identity/sequence STALE and closed movement permission DENIED. Reject bad JSON, duplicate keys, nonfinite numbers and unknown fields; drop malformed frames without extending the current command lease. Duplicate commands may return their earlier receipt but never renew expiry. A stop contact does not wait for this channel.

Clock and recovery rule. The adapter obtains a controller timestamp C between host send H1 and receive H2, giving a signed millisecond offset interval [C-H2,C-H1]; reject a round trip over 10 ms or a mapping over one second old. Translate a host deadline with the lower offset bound minus the measured drift margin, so transport delay cannot create a later deadline. Include controller boot, reject an expired translated deadline and never re-stamp an old command on arrival. Missing/uncalibrated mapping gives UNQUALIFIED and disarms. Link failure gives UNAVAILABLE followed by local expiry; reboot disarms and requires a new mapping/area check. A repaired link only reports status; it cannot replay queued movement.

Call server: D1 completion

Data. Retain D1's Robot, Caregiver, Pairing, and CallAttempt entities and indexes. CallAttempt is durable metadata only; the proposed retention is 30 days, subject to the team's privacy decision. Live socket membership and signaling queues are volatile. New accepted-call participant/epoch bindings are session data, not a second consent database.

Behavior. D1's transaction and terminal-state rules still apply. The server transports invitations/outcomes and authorized signaling, but the robot owns acceptance and capture leases. A robot heartbeat arrives each second; five seconds without one produces offline with last-seen UTC. A restart fails active attempts, resets session baselines, and never redials. Signaling callbacks from old contexts are rejected even if their delivery finishes late. The Node stationary prototype is the current executable bench; the proposed FastAPI product adapter must reproduce its accepted golden traces before replacement. Never run two writable call authorities.

Interface. D1 I1/I2 endpoints remain version 1. The v2 signaling endpoint above is an additive, separate contract; participation requires authenticated call membership. Before signaling, the enrolled robot calls POST /v2/calls/call/binding with JSON action:bind/revoke, context:Context; only it may bind the current accepted context. The server keeps this membership in memory. HTTP 200 returns context:Context, active:bool; exact repeats are idempotent, malformed input gives 400, invalid token 401, wrong role/pairing 403, unknown call 404, conflicting context 409, and identity/storage failure 503. Revocation or socket loss removes the binding; stale revocation cannot remove a newer context. This binding transports the robot authority decision and cannot itself enable capture. Relay unavailability produces failed call, not insecure transport. Database/identity unavailability returns 503, duplicate start handling follows D1, and a terminal attempt cannot return connected. Austin owns the server.

Caregiver iOS app

Data. No persistent media or health observations. Cache account/robot IDs and last known call status with age; stale status is explicitly labeled. An optional refresh credential uses the iOS Keychain and is cleared on sign-out. Access tokens stay in memory and are excluded from application logs. The call database still stores no tokens.

Behavior. Foreground sign-in, paired-robot selection, Call, End, and call-result display form the first iOS workflow. Disable duplicate taps while a request is pending; use one request ID for transport retries of that attempt. Resident acceptance still gates media. Reconnection reads the old outcome and never redials. If the app backgrounds or its foreground session disconnects before background calling is implemented, end the attempt/call rather than imply continued service. Declined/no answer/offline/failed are communication outcomes, not health conclusions. Background incoming-call notifications are not part of the initial companion.

Interface. I16 call(robot:ID) -> Result<CallStatus> and end(call:UUID) -> Result<CallStatus> use I1, with D1 error mappings. I13 uses native authorization-code/PKCE sign-in [3]; I12 uses the same WebRTC/session contract as the robot. Invalid token requires sign-in, 409 refreshes status without a new request, and network/503 errors allow an explicit retry using the same ID. Unknown outcome is shown as unknown, not connected. Use scalable text, VoiceOver labels, and explicit connection/privacy state in the caregiver workflow; validate these separately from resident reach and controls (PR-04). Gavin owns workflow/accessibility, Austin SwiftUI and networking.

Authentication and pairing

Data. The proposed identity provider owns its account credentials; the call database stores only caregiver subject ID and Pairing.active. Client secrets/device credentials live outside the repository in protected deployment/OS stores. No resident CALL/FOLLOW grant is stored as a caregiver role.

Behavior. Use the system authentication session with PKCE and an allowlisted redirect, rather than embedded password collection [3]. The server verifies issuer, intended audience, expiry, subject, and active pairing through the identity adapter [4]. Robot credentials bind a specific robot ID. Sign-out/revocation terminates affected sessions; an account regrant cannot restore resident consent. Recheck authorization at admission and invalidate existing sessions when revocation is learned; provider notification/detection latency is a deployment test, not an instantaneous guarantee.

Interface. I13 signIn() -> Result<AccountSession>; session contains subject:ID, access_token:text (1–4096 characters), and expires_utc:RFC3339 timestamp. I14 verify(token, robot:ID) -> Result<Principal>; Principal is subject ID plus caller, robot, or maintainer role and permitted robot IDs. Maintainer grants metadata-read only, never media or movement. Invalid/expired token maps 401, inactive pairing 403, and provider failure 503. No fallback to trusted fixture role labels in the deployed profile. Austin owns authentication; Keycloak/Authlib remain proposed integrations.

Notifications and visible status

Data. No persistent queue or scheduled call. One active invitation UUID, deadline, announcement state, and confirmed outcome in memory; call outcomes remain in CallAttempt.

Behavior. Announce the named incoming call locally with sound and accessible text within the two-second target. Repeat a provisional sound at most three times, no more than once a second, while the invitation is current; expiry at 60 seconds cancels it. End/decline/privacy cancels future announcements and any callback already queued must recheck context. Caregiver status is pushed only while its foreground authenticated session is current; a missing sound or unavailable screen is surfaced, not treated as acknowledgment.

Interface. announce(context, caller:text, deadline_ms:uint) -> Result<AnnouncementState>; caller is 1–80 characters; state is shown/sounded/canceled/failed/unknown. cancel(context) -> Result<AnnouncementState> is idempotent and returns canceled for a current/already canceled announcement; an old context returns STALE without affecting a newer one. Late invitation gives STALE, unavailable output UNAVAILABLE, bad name INVALID_INPUT. Local handler is resident controls/call manager; Gavin owns wording, Austin delivery. This glue serves US-02/03 rather than adding reminder or emergency scope.

Configuration and deployment profile

Data. Persistent versioned JSON contains revision:ID, endpoint allowlists, calibration/PlatformManifest IDs, drive profile, finite speed/turn caps, age/lease limits, retention and profile:fixture/deployed. The hard caps remain 0.2 m/s and 30 degrees/s; the qualified initial motion cap is at most 0.05 m/s. Motion_ enabled defaults false. EnvironmentPolicy and the StopEnvelope are explicitly version-bound records defined above, not permissive booleans. Credentials are referenced outside the file; no personal data is required. Fixture means synthetic/local test data; deployed means authenticated real-device integration and still requires supervised area approval for the initial prototype.

Behavior. Validate schema, finite values, units, caps, revisions and required credentials on boot. Any missing qualification, stop envelope, capture gate or area guard disables its dependent feature. Apply a safety-related revision only while disarmed with media/tracking capture closed; increment the area revision and invalidate permissions. Do not retain allowed across reboot, relocation or profile changes. Fixture identities/loopback ICE cannot become deployed credentials/routes. No remote model output or caregiver action changes safety configuration. The proposed seven-day diagnostics and 30-day call metadata are defaults to confirm before deployment, not medical-retention rules.

Interface. loadConfig(bytes<=65536) -> Result<Config> and apply(config, expectedRevision:ID) -> Result<ID>. Bad/unknown schema gives INVALID_INPUT, active movement/capture BUSY, revision mismatch STALE, missing profile/calibration/qualification UNQUALIFIED, storage/secret loss UNAVAILABLE. Retain the previous revision for inspection; an invalid current revision cannot start the feature. Austin owns software configuration; Liam approves physical profile/calibration revisions.

Logging and diagnostics

Data. Persistent metadata Event has event UUID (PK), component enum, Context, UTC/local time, event code (admitted/revoked/unavailable/fault/dropped), outcome:ID, and recoverable:bool. Index context plus time for replay. Do not log raw frames, audio, SDP/ICE addresses, tokens, names, or message contents. Provisional retention is seven days with four 5 MB rotated files; CallAttempt's separate 30-day proposal is not a media archive.

Behavior. A nonblocking queue of at most 256 entries drains to bounded storage. On overflow/full disk, drop ordinary diagnostic entries and increment a visible lost-log counter; permission revocation and local stopping never wait on disk. A failed CallAttempt transaction is different: the server returns 503 rather than inventing durable success. Local fault state remains visible in memory even if its diagnostic record cannot be stored.

Interface. record(event) -> Result<Receipt> where Receipt is queued/dropped and an optional event UUID. Invalid event gives INVALID_INPUT; storage loss returns UNAVAILABLE with counter. read(context, limit:uint[1..200]) -> Result<EventBatch> returns {events:Event[], lost:uint} to authenticated metadata maintainers only; return the newest records ordered by time/event UUID, with at most limit entries. Bad limits give INVALID_INPUT, denied access DENIED and storage failure UNAVAILABLE. Austin owns logging; this glue supports US-03 and testable timing/error evidence.

D0 interface audit and revisions

The connection IDs remain stable so D0 can be read beside this design. Context is an immutable record rather than repeating many identity/time arguments. Capture buffers, provider credentials, raw masks, and motor internals stay behind their owners. No published D0/D1 source was overwritten.

Table 1. D0 interface audit

ID	Boundary	D2 decision
I1	Caregiver app / call server	Keep D1 v1 request/outcome contract; tokens in Authorization header, not JSON body. Status is not media or resident consent.
I2	Call server / call manager	Keep D1 invitation/event messages; scope heartbeat sequence to a fresh authenticated session. Add v2 Context only on the separate signaling channel, never silently change v1.
I3	Resident controls / call manager	act(Context, action, optional target) returns a snapshot or typed error. Context removes repeated boot/call/epoch parameters.
I4	Controls / motion	Direct Stop/Privacy, guard and operator-enable contacts override queues. Open/fault revokes the relevant scope; returning closed never rearms. Physical circuit/fixture tests remain required.
I5	Call manager / media	CallLease/null binds current Context, environment revision and capture scopes. Full Privacy revokes the shared platform capture owner, including tracking. Return actual off/unknown state.

ID	Boundary	D2 decision
I6	Call manager / motion	FOLLOW remains separate from CALL, with current area revision and operator enable required. stop is idempotent; status distinguishes requested, settled, fault and unknown.
I7	Tracking / motion	Keep D1 optical-Z output offline. RobotTargetV2 has calibrated forward/right coordinates, uncertainty, Context and expiry; v1 values are never reinterpreted.
I8	Call manager / tracking	Bind explicit target selection to Context and epoch. Reselection after loss is a new action, not automatic acquisition.
I9	Platform / tracking	Expose bounded sensor records and local acquisition timestamps/calibration IDs; hardware carrier details remain private to the adapter.
I10	Platform / media	Use lease-bound device buffers and separate requested/observed camera/mic state. No capture before admission; unknown is not off.
I11	Motion / platform	One typed DriveCommand replaces repeated arguments. Local USB serial carries bounded JSON; conservative clock translation, expiry and the selected caster/balancing response prevent stale travel.
I12	Media / caregiver app	WebRTC media remains separate from authenticated v2 SDP/ICE signaling. Direct/relay configuration is explicit; failed connectivity never selects insecure fallback.
I13	Caregiver app / identity	Native system-session PKCE flow, not raw credentials sent to the team server. App secrets are not embedded in a public native client.
I14	Call server / identity	verify(token, robot) returns a scoped Principal or 401/403/503 mapping. An identity role never grants resident media/movement consent.
I15	Resident / controls	Accessible actions and named-caller/capture/following feedback. Stop/Privacy must remain reachable when the main screen cannot be reached.
I16	Caregiver / iOS app	Foreground Call/End and explicit stale/outcome display; background incoming calls and remote driving remain outside the initial flow.

I1/I2 errors remain the D1 contract. New local records and the signaling/target v2 contracts are explicit design refinements, not a claim that the current Node bench already implements them. D0 and D1 remain frozen comparison artifacts; rationale entries R1–R10 record the changes here.

3 Edge cases and error handling

Each component has one row for each required category. E49–E52 add the received peer-review concerns; E53–E56 close permit, race, schema and dependency failures without renumbering the earlier cases. Recoverable means a later explicit action or repaired dependency may allow a new attempt; it never permits automatic media or movement resume. No row labels a failed physical stop as recoverable

by software alone. Figures 1 and 2 show the corresponding call and movement error paths.

Table 2. Component edge cases and decided handling

ID	Component	Category	Edge case	Design decision, handler, recovery
E01	Resident controls	Boundary values	Seated user cannot reach the main screen	Handler: resident controls. Provide the separate reachable Stop/Privacy fixture; block usability acceptance until the 3/3 reach test passes. Recoverable: after fixture correction.
E02	Resident controls	Timing and concurrency	Stop races with queued Follow or Accept	Handler: resident controls. Prioritize revocation; discard queued old-context actions and show stop requested. Recoverable: new explicit action only.
E03	Resident controls	Bad or missing input	Unknown action, missing target, or stale screen context	Handler: resident controls. Reject with INVALID_INPUT, DENIED, or STALE; refresh the view without changing grants. Recoverable: after valid input.
E04	Resident controls	External dependencies	Touchscreen, speaker, or button circuit fails	Handler: resident controls. Surface output failure/unknown; open safety contact requests stop. Do not show a false acknowledgment. Recoverable: after repair and tests.
E05	Call manager	Boundary values	A second invitation arrives while a call is active	Handler: local authority. Return BUSY; keep the accepted call and reject the competing invitation. Recoverable: after current call ends.
E06	Call manager	Timing and concurrency	End or Privacy occurs during media startup	Handler: local authority. Increment epoch first; late media completions close only their old peer/tracks. Recoverable: fresh invitation and acceptance.
E07	Call manager	Bad or missing input	Missing, expired, or wrong-call lease	Handler: local authority. Deny admission; cancel affected work and retain both permissions false. Recoverable: fresh acceptance.
E08	Call manager	External dependencies	Signaling or authority process disappears	Handler: local authority. Revoke both scopes and close the call; motion enters its local fault response. Recoverable: new authenticated session; motion still qualified separately.

ID	Component	Category	Edge case	Design decision, handler, recovery
E09	Audio/video calling	Boundary values	SDP/ICE exceeds byte or queue limit	Handler: media adapter. Reject/close the attempt with INVALID_INPUT; release buffers and report failed. Recoverable: new valid call.
E10	Audio/video calling	Timing and concurrency	Late ICE/capture result belongs to an ended call	Handler: media adapter. Discard it by Context; never attach tracks or close a newer call. Recoverable: new call unaffected.
E11	Audio/video calling	Bad or missing input	Device permission denied or media format unsupported	Handler: media adapter. Return DENIED/UNAVAILABLE and end the attempt; do not silently enable another capture mode. Recoverable: explicit device permission/configuration change.
E12	Audio/video calling	External dependencies	Camera, microphone, relay, or media link fails	Handler: media adapter. Close outgoing tracks/peer and surface failed or capture unknown; no automatic redial. Recoverable: fresh call after dependency repair.
E13	Person tracking	Boundary values	No instances, empty mask, or too few admitted zones	Handler: tracking adapter. Return unavailable/NO_DEPTH or LOST; withdraw the target and latch loss. Recoverable: explicit reselection after valid evidence.
E14	Person tracking	Timing and concurrency	Duplicate, late, or prior-epoch provider result	Handler: tracking adapter. Reject as STALE; do not extend capture expiry or replace a newer selection. Recoverable: new current capture only.
E15	Person tracking	Bad or missing input	Malformed mask, wrong geometry, NaN, or missing calibration	Handler: tracking adapter. Reject before use; preserve motion_ authorized=false and publish no RobotTargetV2. Recoverable: corrected adapter/calibration and separate validation.
E16	Person tracking	External dependencies	SAM service, Wi-Fi, depth device, or inference worker stops	Handler: tracking adapter. Local watchdog invalidates target without waiting for the service; request stop. Recoverable: fresh selection after repair; no physical-stop guarantee.

ID	Component	Category	Edge case	Design decision, handler, recovery
E17	Motion control	Boundary values	Arrival upper bound reaches 1 m or request exceeds the qualified speed cap	Handler: motion supervisor. Latch zero travel/turn when distance plus uncertainty <=1 m; reject external commands over the measured cap (never >0.2 m/s). Recoverable: fresh valid command after qualification.
E18	Motion control	Timing and concurrency	Stop/expiry races with a target or area update	Handler: motion supervisor/controller watchdog. Latch disarm; caster brakes, balancing keeps zero-travel stabilization in its fixture. Reject old revisions. Recoverable: explicit local/resident rearm after checks.
E19	Motion control	Bad or missing input	Unqualified frame, missing clearance/support, or non-finite target	Handler: motion supervisor. Return UNQUALIFIED/INVALID_INPUT and request local fault response; do not guess clearance. Recoverable: after qualification or valid observation.
E20	Motion control	External dependencies	IMU/encoder/actuator feedback is lost	Handler: local controller/safety path. Latch fault/unknown, isolate drive in the supported test setup; the fixture contains motion/fall. No free-floor balancing test. Recoverable: physical inspection and requalification, not retry.
E21	Robot platform	Boundary values	Voltage, thermal load or clearance reaches its manifest limit	Handler: platform controller. Refuse startup or latch profile stop/fault in the supported fixture. Reject unmeasured limits. Recoverable: inspected hardware and valid measured manifest.
E22	Robot platform	Timing and concurrency	Command/watchdog timeout or reboot occurs during motion	Handler: controller watchdog. Expiry selects the profile stop; reboot isolates drive, changes boot and discards queued commands. Recoverable: inspection, new clock mapping/area check and explicit rearm.
E23	Robot platform	Bad or missing input	Calibration revision, frame units, or sensor sample is missing	Handler: platform adapter. Return UNQUALIFIED/UNAVAILABLE; unknown support/path state is not valid clearance. Recoverable: matching measured configuration.

ID	Component	Category	Edge case	Design decision, handler, recovery
E24	Robot platform	External dependencies	Power/driver/brake fails or fixture readiness is lost	Handler: safety path/operator. Isolate drive; an intact passive fixture contains the test. If the fixture itself fails, report fault/unknown and require manual recovery; never promise containment from failed support. Recoverable: physical repair/requalification.
E25	Call server	Boundary values	One active call already exists for a robot	Handler: call server. Transactional unique index returns 409 ACTIVE_CALL; a matching retry ID returns the original record. Recoverable: after end or same-ID retry.
E26	Call server	Timing and concurrency	Acceptance races with 60-second timeout or server restart	Handler: call server. Terminal state wins; fail stored active attempts on restart and require fresh acceptance. Recoverable: new attempt.
E27	Call server	Bad or missing input	Bad UUID/token fields or an event for another robot/call	Handler: call server. Return 400/401/403/404 as appropriate; do not mutate unrelated attempts. Recoverable: valid authenticated request.
E28	Call server	External dependencies	Database, identity provider, or robot heartbeat is unavailable	Handler: call server. 503 for unverifiable/storage failure; offline after five seconds without heartbeat with last-seen time. Recoverable: repair plus explicit retry/new session.
E29	Caregiver iOS app	Boundary values	Repeated Call taps while a request is pending	Handler: iOS session adapter. Use one request ID and disable duplicate dispatch; show pending until a real outcome. Recoverable: existing attempt or explicit later call.
E30	Caregiver iOS app	Timing and concurrency	App backgrounds, disconnects, or reconnects after End	Handler: iOS session adapter. End the foreground call; on reconnect fetch outcome only, never redial. Recoverable: new foreground Call.
E31	Caregiver iOS app	Bad or missing input	Token expired or server returned an unknown/malformed status	Handler: iOS session adapter. Require sign-in or show unavailable/unknown; never display connected from incomplete data. Recoverable: valid session/compatible response.

ID	Component	Category	Edge case	Design decision, handler, recovery
E32	Caregiver iOS app	External dependencies	No network, phone media permission, or relay route	Handler: iOS session adapter. Surface the specific failure; permit explicit same-ID retry where D1 allows it. Recoverable: after dependency restoration.
E33	Authentication and pairing	Boundary values	Account exists but no active pairing permits this robot	Handler: identity adapter. Return 403; do not create a pairing or resident grant implicitly. Recoverable: explicit authorized pairing.
E34	Authentication and pairing	Timing and concurrency	Revocation or logout races with an in-flight request	Handler: identity adapter. Invalidate the learned session/epoch before admitting new work; old callbacks cannot restore it. Recoverable: fresh sign-in and new resident acceptance.
E35	Authentication and pairing	Bad or missing input	Wrong issuer/audience, expired token, or bad PKCE callback	Handler: identity adapter. Return 401 or abort sign-in; discard credentials without logging them. Recoverable: fresh legitimate sign-in.
E36	Authentication and pairing	External dependencies	Identity discovery/introspection service is unavailable	Handler: identity adapter. Return 503 and deny new access; fixture roles are never a production fallback. Recoverable: explicit retry after recovery.
E37	Notifications	Boundary values	Invitation has already expired or sound repeat cap is reached	Handler: notification adapter. Cancel announcement; report no answer/expired without repeating indefinitely. Recoverable: new invitation.
E38	Notifications	Timing and concurrency	Decline/End/privacy occurs after a timer callback is dequeued	Handler: notification adapter. Recheck Context at dispatch and cancel stale sound/banner. Recoverable: new current invitation.
E39	Notifications	Bad or missing input	Caller name, deadline, or call identity is missing	Handler: notification adapter. Reject announcement with INVALID_INPUT; do not display an unnamed trusted caller. Recoverable: corrected authenticated invitation.
E40	Notifications	External dependencies	Screen/audio output or foreground status transport fails	Handler: notification adapter. Report failed/unknown delivery to call manager; an unobserved alert is not acknowledgment. Recoverable: output repair or fresh session.

ID	Component	Category	Edge case	Design decision, handler, recovery
E41	Configuration	Boundary values	Speed/lease/retention value exceeds the permitted schema range	Handler: configuration loader. Reject the new revision and keep the dependent feature disabled. Recoverable: validated revision.
E42	Configuration	Timing and concurrency	Configuration changes while a call or movement is active	Handler: configuration loader. Refuse apply with BUSY; require disarm/media close and expected-revision check. Recoverable: after idle and revision refresh.
E43	Configuration	Bad or missing input	Unknown keys, invalid JSON, missing units or calibration ID	Handler: configuration loader. Return INVALID_INPUT/UNQUALIFIED; do not substitute permissive defaults. Recoverable: corrected complete config.
E44	Configuration	External dependencies	Secret store, manifest file, or approved endpoint is missing	Handler: configuration loader. Return UNAVAILABLE/UNQUALIFIED; do not enter deployed/motion mode. Recoverable: restored approved dependencies.
E45	Logging	Boundary values	256-entry queue or rotated-file quota is full	Handler: diagnostic writer. Drop ordinary metadata with a visible lost-log counter; never block revoke or local control. Recoverable: after drain/storage recovery; lost records remain lost.
E46	Logging	Timing and concurrency	Late log event arrives from an old call or boot	Handler: diagnostic writer. Store only its original Context as historical metadata; never mutate live call state. Recoverable: live operation unaffected.
E47	Logging	Bad or missing input	Event includes a token, raw media, name, or invalid code	Handler: diagnostic writer. Reject/redact at the typed boundary with INVALID_INPUT; increment diagnostic rejection count. Recoverable: valid metadata-only event.
E48	Logging	External dependencies	Disk is full/locked or writer unavailable	Handler: diagnostic writer. Report diagnostic degradation and retain in-memory fault status; call-database failure still returns 503 separately. Recoverable: storage repair; no fabricated durable receipt.

ID	Component	Category	Edge case	Design decision, handler, recovery
E49	Robot platform	Boundary values	Low profile is missed, protrudes into an approach, or obstructs a walkway (PR-01)	Handler: platform qualification, Liam with Gavin. Refuse the physical revision/placement until visibility, stability, clearance and parking checks pass. Recoverable: revised mockup and requalification; not a software retry.
E50	Motion / local authority	External dependencies	Bathroom/wet-area guard opens, faults or cannot establish the approved route (PR-02)	Handler: local authority plus wired guard. Publish excluded/unknown; revoke FOLLOW and all capture; latch stop. Physical barriers define the first route. Recoverable: local inspection, new area approval and explicit acceptance/rearm.
E51	Media / local authority	Timing and concurrency	Privacy guard opens during a call or capture close is uncertain (PR-03)	Handler: wired capture latch/platform broker. Drop capture enable, invalidate epoch and clear media/tracking buffers; display off only with driver/gate confirmation. Recoverable: inspected guard/devices and fresh area/call approval.
E52	Resident controls / iOS app	Bad or missing input	User cannot identify the action/state, or touch/voice input is unsuitable (PR-04)	Handler: UI adapters, Gavin/Austin. Retain grants unchanged; offer plainly labeled accessible controls and reachable switch alternative, surface uncertainty. Recoverable: supported input and explicit action; usability acceptance waits for task checks.
E53	Local authority	Boundary values	Area permit reaches its expiry, including exact equality	Handler: authority timer. At now>=expiry increment revision/epoch and revoke capture/FOLLOW, even without a callback. Recoverable: a fresh local check and new resident actions.
E54	Local authority	Timing and concurrency	Stale armArea result races with relocation, config change or guard opening	Handler: local authority. Atomically compare boot/policy/revision and physical guard before publishing allowed; stale completion cannot restore access. Recoverable: new local check.
E55	Platform / configuration	Bad or missing input	Malformed drive frame, unknown profile, or absent clock/stop-envelope data	Handler: typed adapter/configuration loader. Reject with INVALID_INPUT or UNQUALIFIED; retain disarm; never extend an old lease. Recoverable: corrected record and qualification.

ID	Component	Category	Edge case	Design decision, handler, recovery
E56	Platform / local authority	External dependencies	Guard/capture feedback or USB link is unavailable	Handler: wired guard and controller watchdog. Drop capture enable, expire movement locally and show unknown where unobserved. Recoverable: inspection, fresh mapping/area approval and explicit new grants.

4 Requirements traceability table

The baseline is the five user stories, their use-case acceptance criteria, and D0's explicit platform/privacy constraints. Derived constraints are labeled D-NF rather than presented as new stakeholder promises. D-NF10–13 are peer-derived design constraints linked to the received feedback; their validation is still open. Must means required for the scoped journey; proposed limits/retention still require confirmation. A design mapping is not a passing verification result. Open cells below explain a gap rather than claiming a feature is complete.

Table 3. Forward requirements traceability

Requirement and source	Priority	Specific design elements / error IDs	How it is addressed; remaining gap
US-01 - Continue a call during a requested room transition	Must	CallLease; RobotTargetV2; command lease; Fig. 1 and 2; E06, E14, E18	Independent CALL/FOLLOW and local target expiry preserve call state during tracking loss. Live tracking and physical route behavior remain unqualified.
US-02 - Accept or decline before camera/microphone turn on	Must	act(accept/decline); setMedia(CallLease); Fig. 1; E02, E06, E10, E34	No lease before current acceptance; epoch revocation precedes async cleanup. Actual device-off behavior still needs fixture observation.
US-03 - Understand declined, no answer, and robot offline outcomes	Must	D1 CallAttempt.status/last_seen; I1 CallStatus; Fig. 1; E25–E28, E31	Terminal outcomes and explicit stale labels describe communication, not health. Foreground iOS integration remains to be built.
US-04 - Do not switch to or follow another person	Must	Selection epoch; admit; RobotTargetV2; Fig. 2; E13–E16, E19	Ambiguity/loss latches disarm. Same-box identity swaps and sustained real identity continuity remain unresolved, so no motion admission.

Requirement and source	Priority	Specific design elements / error IDs	How it is addressed; remaining gap
US-05 - Reach Stop/Privacy from a seated position	Must	I4 supervised contacts; I15 ResidentView/control fixture; Fig. 2; E01, E04, E21	Gavin/Liam will measure reach and button operation in 3/3 attempts. No tested mounting position or switch geometry is claimed.
AC-01.1 - Named caller within 2 s; no capture before acceptance	Must	announce; act; CallLease; Fig. 1; E06, E10, E37–E40	Announcement and capture admission are distinct; two-second end-to-end timing and physical capture indicators remain unverified.
AC-01.2 - 5 m transition; <=0.2 m/s; stop within 1 m; keep call	Must	Qualified speed cap; follower; DriveCommand; I12; Fig. 2; E17–E20, E55	Initial cap is 0.05 m/s and must satisfy the measured stopping bound. The 5 m path, turning and concurrent media remain physical integration tests.
AC-01.3 - Loss: zero travel request <=1 s; <=25 mm final displacement; <5 mm/s for 1 s within 10 s; no automatic resume	Must	StopEnvelope inequality; profile stop; stop anchor/window; Fig. 2; E14, E18–E20, E22	Full loss-to-brake delay and measurement margin limit speed. Settled is observed, not assumed. Each base must qualify separately; free-floor balancing remains unavailable.
AC-02.1 - Caregiver sees Connected within 2 s of acceptance	Must	D1 MEDIA_CONNECTED transition; v2 signal; I1 status; Fig. 1; E10–E12, E28	Only confirmed media yields connected. Bench 30-second deadlines do not prove this target; concurrent deployment test needed.
AC-02.2 - Declined within 2 s; camera/microphone never started	Must	act(decline); no CallLease; cancel announcement; Fig. 1; E06, E10, E38	Decline is terminal; late callbacks are rejected. Notification latency and physical capture audit remain open.
AC-02.3 - After 5 s without robot heartbeat show offline and last seen	Must	D1 Robot.last_seen/last_ sequence; I2; I1; Fig. 1; E26, E28, E30	Session-scoped heartbeat monotonicity drives offline outcome. New session never resumes an old attempt; real fault-injection test needed.
UC-01 - Camera/mic off, FOLLOW off, robot stopped after End	Must	act(end); setMedia(null); setFollow(false); Fig. 1 and 2; E06, E10, E18, E20	Revoke permissions immediately; report observed capture/movement separately. Physical stop and capture-state checks remain unqualified.

Requirement and source	Priority	Specific design elements / error IDs	How it is addressed; remaining gap
UC-01/02 - One floor, invited paired caregiver, no active second call	Must	Pairing.active; unique active-call index; Context; Fig. 1; E05, E25, E33	Busy arbitration and pairing protect scope. Route accessibility/clearance still require measured configuration.
D-NF01 - No audio/video recording; minimal retained metadata (D0 privacy, UC-01)	Must	CallAttempt metadata only; record(Event); Fig. 1; E09, E45, E47–E48	Buffers/signaling are volatile; provisional seven-day diagnostics and 30-day call metadata need team approval, not medical retention assumptions.
D-NF02 - Local Stop/Privacy and movement authority (D0 decisions 4/5)	Must	I4 guard/enable/stop; epoch; controller watchdog; Fig. 1/2; E02, E06, E18, E20, E56	Wired revocation and local profile response do not wait on a network. Circuit behavior and stopping distance require supported tests.
D-NF03 - Observation freshness <=200 ms (D1 development constraint)	Must	Capture ledger; target expiry; StopEnvelope; Fig. 2; E14, E16, E18, E55	Expiry rejects late work; the stopping bound includes sensor age plus downstream delay. A fresh record alone never qualifies control stability.
D-NF04 - Authenticated encrypted remote access (D0 I12–I14)	Must	PKCE signIn; verify; Principal/Pairing; v2 signal; Fig. 1; E11–E12, E33–E36	Fixture identities/loopback routing cannot be deployed. Provider credentials, TLS/relay route, native media build, and revocation latency need verification.
D-NF05 - No media or movement resume on reconnect/restart (D0 decision 5)	Must	Context boot/epoch; terminal CallAttempt; new peer; Fig. 1 and 2; E06, E10, E18, E22, E26, E30	Old timers/results cannot restore leases; a fresh explicit call, acceptance, and Follow are required.
D-NF06 - Custom build, fit, power and mechanical testing (D0 decision 1)	Must	PlatformManifest.profile; profile adapter; fixture; Fig. 2; E21–E24, E55	Both caster and balancing candidates are explored; no final base is selected. Liam owns CAD/build/fixtures and separate qualification; BOM/integration remain open.
D-NF07 - Affordable hardware and hosting within approved allowance (D0 constraints)	Must	Versioned Config; PlatformManifest; deployment profile; Fig. 2; E21, E41, E44	No numeric budget is supplied. Quote hardware/carrier/power, identity host, relay and operation; architecture cannot claim budget compliance yet.

Requirement and source	Priority	Specific design elements / error IDs	How it is addressed; remaining gap
D-NF08 - Accessible readable, honest current state (D0 I15/16, US-05)	Must	ResidentView; AnnouncementState; iOS CallStatus age; Fig. 1; E01, E04, E31, E40	Use readable state labels and unknown where unobserved. Reach/VoiceOver/text sizing and actual display testing remain open.
D-NF09 - Traceable diagnostic evidence without blocking controls (D0 error handlers; derived)	Must	Event context/time index; bounded record queue; Fig. 1 and 2; E45–E48	Metadata supports call/error/timing reconstruction; queue/disk failure never grants consent or blocks revocation. No telemetry log is proof of physical success.
D-NF10 - Conspicuous form and clear walking routes (peer PR-01)	Must	PlatformManifest visibility revision; approved parking; Fig. 2; E21, E49	Liam/Gavin compare full-scale mockups and clearance/stability trade-offs. Final dimensions and physical visibility/approach tests remain open.
D-NF11 - Exclude bathrooms/wet areas from initial operation (peer PR-02)	Must	armArea; EnvironmentPolicy/state; I4 guard; Fig. 2; E19, E50, E53–E56	Physical barriers, local inspection, a bounded permit and held operator enable define supervised trials. No autonomous bathroom detection or unattended readiness is claimed.
D-NF12 - No bathroom camera exposure or unapproved listening (peer PR-03)	Must	View check; shared capture owner/latch; epoch revoke; Fig. 1/2; E06, E12, E51, E54, E56	Opaque guards exclude bathroom views; faults revoke media and tracking capture. Physical closure/feedback and full-route view checks remain to be tested.
D-NF13 - Low-effort controls across varied abilities/technology familiarity (peer PR-04)	Must	ResidentView/act/I4; scalable iOS text/VoiceOver; Fig. 1; E01, E04, E52	Test the key call/Stop/Privacy tasks, state recognition, errors and assistance. Reach and formative evaluation are pending; universal usability is not claimed.

Reverse check

Every D0 and glue component below has a requirement and a concrete design element. Optional routines, help detection, smart-home control, item handling, and screen mechanisms are not additional requirements in this baseline. No component is justified merely because a library exists.

Table 4. Reverse architecture-to-requirement check

Component	Requirement basis	Concrete design / figure	Edge cases
Resident controls	US-02/05; D-NF08/13	ResidentView, act, supervised I4 contacts; Fig. 1/2	E01–E04, E52
Call manager	US-01/02/04; D-NF02/05/11/12	CallLease, armArea, epoch and permit revision; Fig. 1/2	E05–E08, E50/51, E53/54
Audio/video calling	US-01/02; AC-02.1/02.2; D-NF12	lease-bound capture broker/latch, close; Fig. 1	E09–E12, E51/56
Person tracking	US-04; D-NF03	capture ledger, admit, RobotTargetV2; Fig. 2	E13–E16
Motion control	US-01/04; AC-01.2/01.3; D-NF11	StopEnvelope, profile stop, DriveCommand; Fig. 2	E17–E20, E50/55
Robot platform	US-05; D-NF06/07/10/12	PlatformManifest, shared capture owner, USB adapter; Fig. 2	E21–E24, E49, E55/56
Call server	US-03; AC-02.3	CallAttempt, Robot.last_seen; Fig. 1	E25–E28
Caregiver iOS app	US-01/03; D-NF08/13	Call/End, readable accessible status; Fig. 1	E29–E32, E52
Authentication	US-02/03; D-NF04	Principal, Pairing, PKCE signIn/verify; Fig. 1	E33–E36
Notifications	US-02/03; AC-01.1	announce/cancel, delivery observation; Fig. 1	E37–E40
Configuration	D-NF02/05/06/07/11/12	loadConfig/apply; EnvironmentPolicy/StopEnvelope; Fig. 2	E41–E44, E50/51, E55
Logging	US-03; D-NF01/09	Event, record, bounded diagnostics; Fig. 1/2	E45–E48

5 Design rationale

D0's numbered decisions remain the starting rationale. The entries below record refinements, not a replacement D0 log. R8 and R9 are marked peer-review-driven and cite PR-01–04; R7/R10 record the completed stop/profile design; the other entries are engineering refinements. D0 and D1 have not been edited for this work. Peer review took place on October 26, 2026. Follow-up dates are proposed targets; physical and usability validation are not recorded as completed.

R1. One local permission authority. We chose one local authority with bounded adapters over separate writable permission state in the app, server, and robot because revocation must have one current meaning. We accepted the trade-off of explicit boot/epoch propagation because it makes stale work rejectable.
D0 decisions 4/5; D1 consent boundary; US-02, D-NF02/05; E06, E10, E18. Engineering refinement; not a peer-driven change.

R2. Continue the tested bench before porting. We chose contract-preserving adapters around the working Node/WebRTC bench over an immediate FastAPI rewrite because eight local generated-media calls provide a finite regression baseline. We accepted the trade-off of retaining a temporary implementation different from D1 because golden traces can test a later product port without two authorities.
D0 decision 3; D1 backend choice; US-01/03; E26, E28. D1 is a proposal, not a claim about the current runtime. Engineering refinement; not a peer-driven change.

R3. Explicit robot-frame target type. We chose RobotTargetV2 over renaming D1 optical depth as robot distance because camera depth, radial ToF range, and floor-plane position are different quantities. We accepted the trade-off of a new adapter and major-version type because a wrong frame can make a valid-looking number unsafe to use.
D0 I7/I9; D1 TrackRecord; US-04, D-NF03; E15, E19. D0/D1 stay unchanged; new type is specified here. Engineering refinement; not a peer-driven change.

R4. SAM provider behind conservative admission. We chose to evaluate SAM instance masks behind the existing capture/selection/depth gates over treating provider object IDs as permission to follow because responses can be late and object IDs do not establish cross-request personal identity. We accepted the trade-off of abstention and possible unusable latency because a mask demo must not imply qualified motion.
D0 decisions 4/5; D1 tracking baseline; US-04; E13–E16. Older detectors remain comparison evidence; provider/live transport is unresolved. Engineering refinement; not a peer-driven change.

R5. Separate media signaling from outcome records. We chose a bounded v2 SDP/ICE channel alongside D1 call-status APIs over putting provider/media buffers into CallAttempt because signaling is transient and has different limits and privacy concerns. We accepted the trade-off of another narrowly scoped endpoint because it keeps the durable data model minimal.
D0 decision 3 and I12; D1 data model; US-01/03, D-NF01/04; E09–E12, E27. Engineering refinement; not a peer-driven change.

R6. Foreground iOS first. We chose the caregiver iOS companion with foreground Call/End over adding background incoming-call infrastructure now because the requested companion can exercise the scoped journey without claiming untested ringing behavior. We accepted the trade-off of ending on background/disconnect because reliable background media and notifications need their own integration.
D1 iOS choice; US-01/03; E29–E32. No new background-calling requirement is introduced. Engineering refinement; not a peer-driven change.

R7. Profile-specific stopping with measured admission. We chose explicit caster braking or supported zero-travel balancing, bounded by a measured StopEnvelope, over treating zero requested speed as a physical stop because the two bases react differently to torque removal and feedback faults. We accepted the trade-off of a lower initial speed cap and supported testing because the 25 mm target must include sensing delay, command expiry and real braking distance.
D0 decision 4; D1 stopping evidence; AC-01.3, D-NF02/03/06; E18–E24, E55/56. Engineering completion; no physical pass is claimed.

R8. Visibility and accessible controls before added mechanisms (peer-review-driven: PR-01/04). We chose full-scale visibility, reach and usability comparisons with a fixed screen/mount baseline over immediately adding a telescoping screen or simply increasing height because a conspicuous form and reachable controls must also preserve stability and walkway clearance. We accepted the trade-off of keeping dimensions and household use unapproved because Liam and Gavin need measured mechanical checks and task observations before selecting the final geometry.
D0 decisions 1/2/6; US-05, D-NF06/07/08/10/13; E01, E21–E24, E49/E52. Updated D2 platform and controls; physical testing remains open. D0/D1 unchanged.

R9. Supervised bathroom exclusion and shared capture revocation (peer-review-driven: PR-02/03). We chose a locally checked, guarded trial route with an expiring permit and one capture owner over inferring bathroom safety from a room label or closing only the video-call camera because the first prototype has neither reliable autonomous boundaries nor independent permission for tracking capture. We accepted the trade-off of physical barriers, an operator and no bathroom monitoring because faults must have an explicit restrictive outcome without creating unapproved listening or filming.
D0 decisions 4/5 and privacy constraint; D-NF11/12; E50/51, E53–E56; Fig. 1/2. Local/wired behavior is defined; capture/guard performance remains a hardware test. D0/D1 unchanged.

R10. Explore both bases behind one contract. We chose explicit caster and balancing profiles behind DriveCommand/MotionObservation over selecting a final base prematurely because the team is exploring both a two-drive-wheel/rear-caster platform and a balancing platform. We accepted the trade-off of separate manifests, fixtures and qualification work because a successful stop or simulation result on one cannot establish safe behavior on the other.
D0 decisions 1/4; D-NF06/07; E20–E24, E55; Fig. 2. Records the current exploration rather than changing the shared goal or claiming a hardware selection. D0/D1 unchanged.

Technical references

[1] WebRTC peer connections and separate signaling; [2] Meta SAM output, frame association, and object-ID limits; [3] RFC 8252, native-app authorization and PKCE; [4] Keycloak OIDC endpoints and token verification.

6 Peer design review notes

Status: feedback recorded; follow-up validation open. Clare Hooper (mirror project) raised three feedback themes during the review on October 26, 2026: the robot may be too low to notice and could become a trip hazard; bathroom slip/injury concerns conflict with camera privacy; and ease of use should be a primary focus for people with differing technology familiarity. The paraphrases below split the bathroom theme into safety and privacy actions. No testing or completion by reviewers is implied.

Feedback coverage. These notes record the feedback from Clare Hooper reported by Austin. Our team raised one issue while reviewing her mirror project; no second issue was raised.

Schedule. The written design revisions below were completed on November 11, 2026. November 18, 2026 is the proposed verification follow-up target, aligned with the presentation-materials milestone. The final design report is due December 2, 2026. The proposed action targets are planning dates, not evidence that physical or usability checks have passed.

PR-01. Low profile and trip risk

Criterion: risk. **Issue/source:** the robot may be too short to notice, creating a trip hazard; raised by Clare Hooper (mirror project). **Action:** define visibility, stability, clearance, reach and parking criteria for both base candidates, with full-scale mockup comparisons before dimensions are approved. **Owner:** Liam Patel, with Gavin Lamphier. **Priority:** must-fix. **Due:** design revision November 11, 2026 (applied); verification follow-up November 18, 2026 (proposed). **Status:** visibility/parking design updated; compare mockups and physically check both base candidates. **Links:** Robot platform; D-NF10; E49; R8; Fig. 2.

PR-02. Bathroom access and slip risk

Criterion: risk. **Issue/source:** bathrooms are a high-risk setting for slips/injury, and the robot could add an obstruction; raised by Clare Hooper (mirror project). **Action:** specify a physically guarded route, local armArea/expiry and profile-specific stopping; define supported tests for guard loss and boundary uncertainty before movement is admitted. **Owner:** Liam Patel, with Austin Heiss. **Priority:** must-fix. **Due:** design revision November 11, 2026 (applied); verification follow-up November 18, 2026 (proposed). **Status:** guarded trial/armArea/expiry design specified; physical guard and stopping tests remain open. **Links:** Local authority/motion; D-NF11; E50, E53–E56; R9; Fig. 2.

PR-03. Bathroom camera privacy

Criterion: risk. **Issue/source:** camera monitoring in or toward the bathroom conflicts with privacy; raised by Clare Hooper (mirror project). **Action:** specify full-route bathroom view exclusion, shared media/tracking capture revocation and hardware gate behavior; define shutdown/off/unknown checks. No ambient microphone monitoring is substituted. **Owner:** Austin Heiss, with Gavin Lamphier. **Priority:** must-fix. **Due:** design revision November 11, 2026 (applied); verification follow-up November 18, 2026 (proposed). **Status:** shared capture gate and revocation specified for media/tracking; physical closure/view tests remain open. **Links:** Platform capture/local authority; D-NF12; E51, E54, E56; R9; Fig. 1/2.

PR-04. Simple, accessible interaction

Criterion: clarity. **Issue/source:** prioritize ease of use for residents with varying technology experience and abilities; raised by Clare Hooper (mirror project). **Action:** specify plainly labeled call/Stop/Privacy controls, readable non-color-only state, reachable alternate input and caregiver text-size/VoiceOver support; define the accept/decline/Stop/Privacy/state-recognition walkthrough and error/assistance observations. **Owner:** Gavin Lamphier, with Austin Heiss. **Priority:** must-fix. **Due:** design revision November 11, 2026 (applied); verification follow-up November 18, 2026 (proposed). **Status:** interaction criteria added; formative evaluation open. **Links:** Resident controls/iOS; D-NF13; E52; R8; Fig. 1.

Feedback summary. Risk drew the most recorded actions: three of four; clarity accounts for one. These counts describe the action items, not numbers of reviewers. Review-driven D2 changes appear in R8/R9 and the linked component, error, and traceability sections. D0/D1 remain unchanged.

Our reviewing. One issue was raised (feasibility), by Austin on Clare Hooper's mirror project: the AI clothing try-on should accurately represent how garments fit the user's body rather than guess their appearance. No second issue was raised.

Follow-up status. The received feedback, reviewer, review date and single outgoing issue are recorded. Owners should review the proposed November 18 targets and record evidence for the remaining physical, privacy and usability checks. The design actions now define mechanisms, interfaces and fault behavior; fabrication, deployment and test evidence remain open.